
RL - Reinforcement Learning

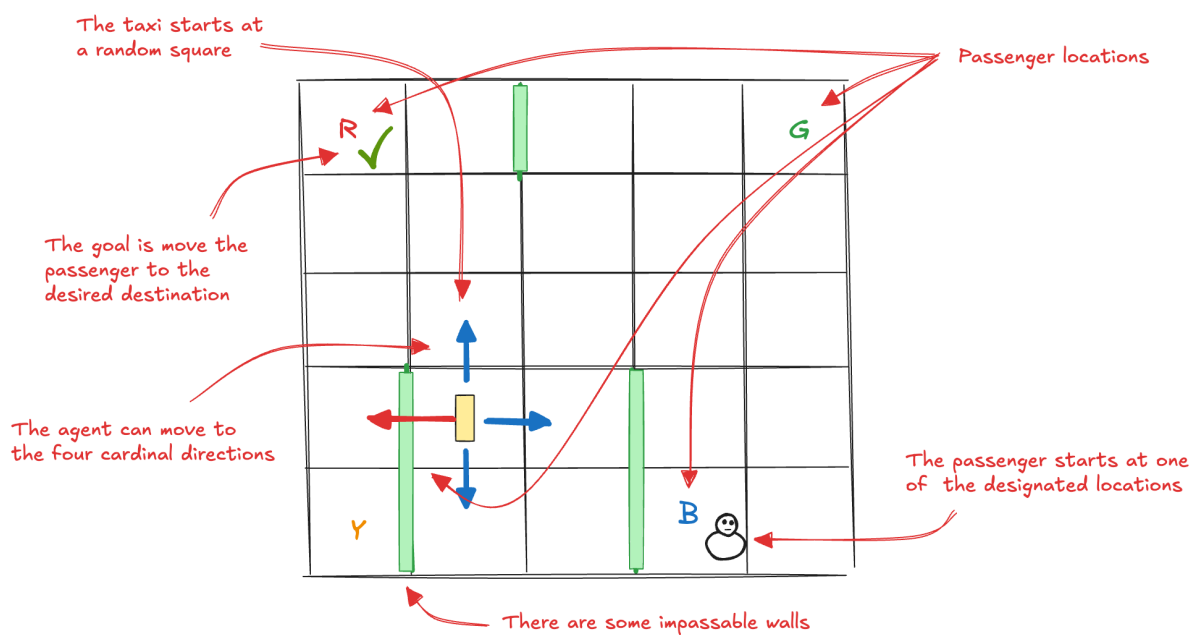
Solutions

Riccardo Berta

2026.05.20

1 Taxi

The Taxi problem involves navigating passengers in a grid world, picking them up and dropping them off at one of four locations (Red, Green, Yellow and Blue) in a 5x5 grid world. The taxi starts off at a random square and the passenger at one of the designated locations. The goal is move the taxi to the passenger's location, pick up the passenger, move to the passenger's desired destination, and drop off the passenger. Once the passenger is dropped off, the episode ends.



There is six possible actions. The agent can move to the four cardinal directions (south, north, east or west) and two other possible actions which are the pick-up or the drop-off:

- 0: Move south (down)
- 1: Move north (up)
- 2: Move east (right)
- 3: Move west (left)
- 4: Pickup passenger
- 5: Drop off passenger

The state is described by the taxi location on the grid (a row and a column number between 0 and 4), a location to drop-off the passenger from four choices, and the passenger which can be in one of the four locations or inside the taxi, in total 500 discrete states. An observation is returned as an int that encodes the corresponding state, calculated by: $((\text{taxi_row} * 5 + \text{taxi_col}) * 5 + \text{passenger_location}) * 4 + \text{destination}$

The player receives positive rewards for successfully dropping-off the passenger at the correct

location. Negative rewards for incorrect attempts to pick-up/drop-off passenger and for each step where another reward is not received.

1 - Import the 'Taxi-v3' grid world from Gymnasium:

```
# You can get the environment from Gymnasium in the same way we got 'Frozen Lake';  
# in order to visually plot the environment you can import it  
# using render_mode="rgb_array"  
  
import gymnasium as gym  
taxi = gym.make('Taxi-v3', render_mode="rgb_array")
```

2 - Create a random policy as a baseline:

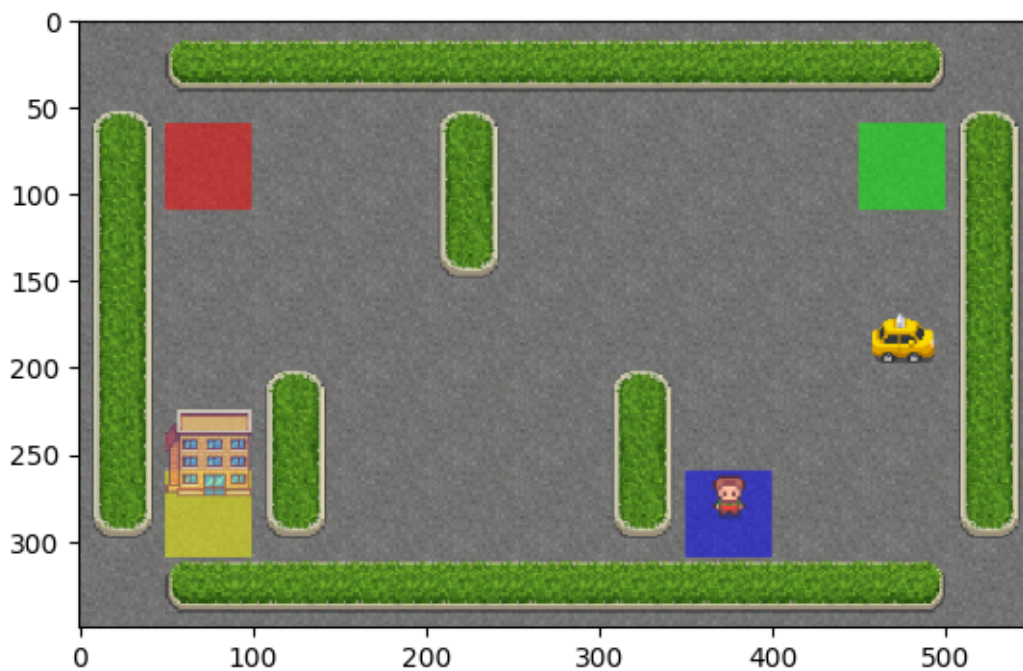
```
# You have to create a function that get in input a state and provide a random action  
# (in the range [0;5])  
  
import numpy as np  
  
def pi_random(s):  
    return np.random.choice(tuple(taxi.unwrapped.P[0].keys()));
```

3 - Show the policy in action by rendering the environment several times after different decisions from the random policy:

```
# You have to use a while loop in order to provide the current state to  
# the policy and then make a step in the environment using the action  
# provided by the policy.  
# Try to create a function (to be called "show_policy")  
# to be reused with other policies later.  
# Hints: env.render() provides you an array representing  
# an image of the environment; plt.imshow() can be used to visualize  
# the image on the screen.  
  
import time  
import matplotlib.pyplot as plt  
from IPython.display import clear_output  
  
def show_policy(env, pi, n_episodes=1, max_steps=50):  
    for _ in range(n_episodes):  
        state, info = env.reset()
```

```
done = False
steps = 0;
while not done and steps < max_steps:
    action = pi(state);
    state, reward, terminated, truncated, info = env.step(action)
    if(terminated or truncated):
        done = True;
    clear_output(wait=True)
    plt.imshow(env.render())
    plt.show()
    steps += 1
    print('Step :', steps, 'Action: ', action)
    time.sleep(0.5)

show_policy(taxi, pi_random)
```



Step : 50 Action: 5

4 - Write a brute-force function in order to evaluate the probability of success and the average return obtained by a policy, then evaluate the random policy:

```
# You can reuse the "evaluate" function that we already apply to
# the Frozen Lake environment, however pay attention on how to determine
# the end of an episode (it is no more state 15).
# Hints: you have the "done" information from step function.

def evaluate(env, pi, n_episodes=1000):
    success = 0;
    returns = []
    for _ in range(n_episodes):
        done = False;
        steps = 0;
        state, info = env.reset();
        returns.append(0.0)
        while not done:
            state, reward, terminated, truncated, info = env.step(pi(state))
            if(terminated or truncated):
                done = True;
                returns[-1] += reward;
                steps += 1;
            if terminated:
                success += 1;
    return (success/n_episodes)*100, np.mean(returns);

probability_success, mean_return = evaluate(taxi, pi_random);

print("Reaches goal ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);
```

Reaches goal 5.1 %

Obtains an average undiscounted return of -769.398

5 - Use the **value-iteration algorithm** to calculate the optimal policy and also the optimal state-value function:

```
# You can reuse the "value_iteration" function that we already
# apply to the Frozen Lake environment.

def value_iteration(env, gamma=0.99, theta=1e-10):

    # we need to unwrap the environment to access its
    # internal dynamic
```

```

dynamics = env.unwrapped.P;

# initialize a state-value function
v = np.zeros(len(dynamics))

while True:
    q = np.zeros((len(dynamics), len(dynamics[0])))

    # for every transition of every action in every state, we ...
    for s in range(len(dynamics)):
        for a in range(len(dynamics[s])):
            for prob, next_state, reward, done in dynamics[s][a]:
                # ... calculate the action-value function
                q[s][a] += prob * (reward + gamma * v[next_state])

    # After each sweep over the state space, make sure the state-value function
    # keeps changing,
    # otherwise, we found the optimal state-value function and should break out
    if np.max(np.abs(v - np.max(q, axis=1))) < theta:
        break;

    # we don't need a separate policy-improvement phase
    v = np.max(q, axis=1)

    # only at the end, we extract the optimal policy
    def pi(s):
        return np.argmax(q[s])

    return v, pi

v_best, pi_best = value_iteration(taxi, gamma=0.99)

```

6 - Calculate the performance of the obtained optimal policy using the brute force approach:

```

# You can reuse the "evaluation" function written before,
# in order to evaluate the optimal policy

probability_success, mean_return = evaluate(taxi, pi_best);
print("Reaches goal ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);

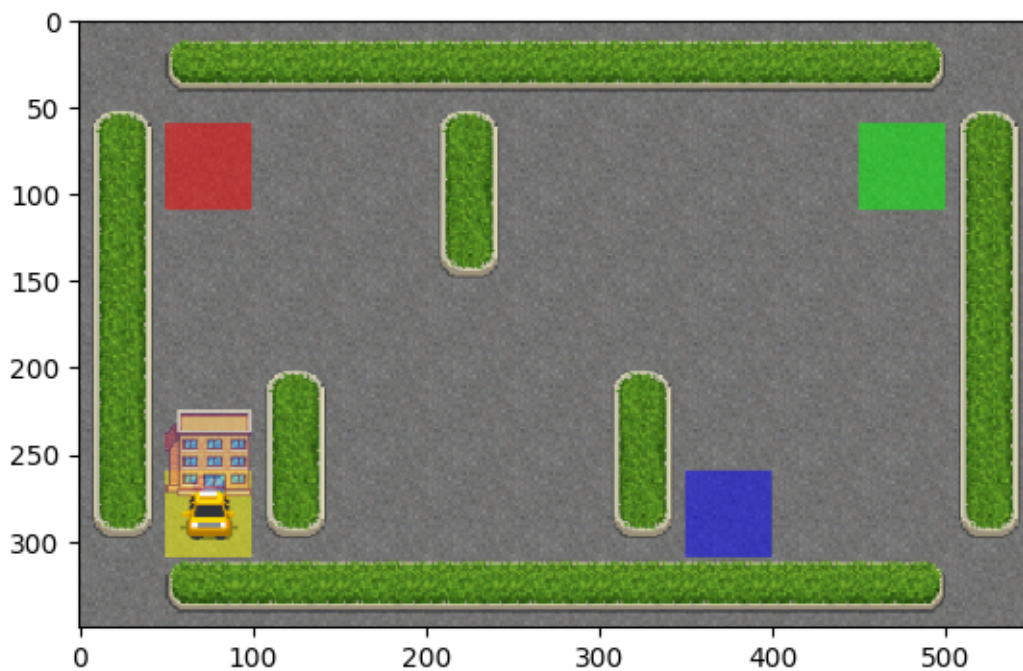
```

Reaches goal 100.0 %

Obtains an average undiscounted return of 7.97

7 - Show the optimal policy:

```
# You can reuse the "show_policy" function written before,  
# in order to show the optimal policy  
  
show_policy(taxi, pi_best, n_episodes=5)
```



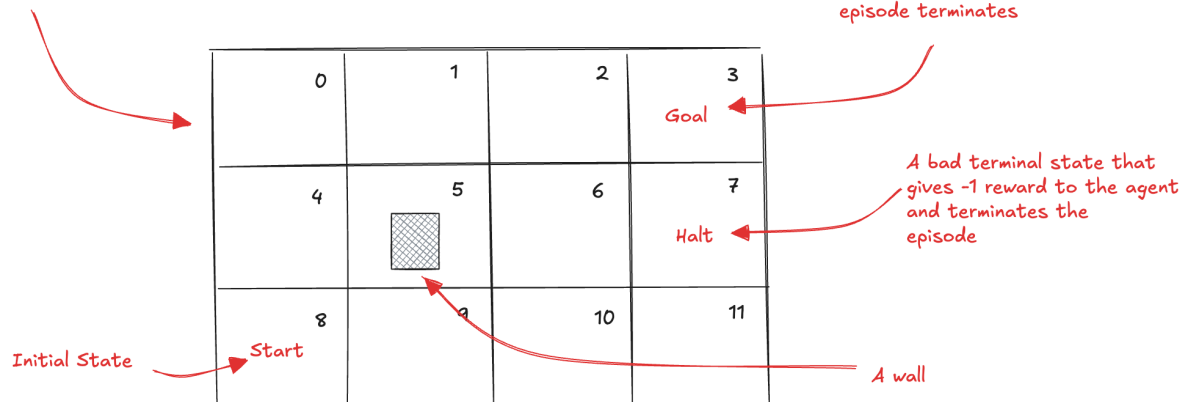
Step : 14 Action: 5

2 Gridworld

Create the environment from the Russell and Norvig's book on AI: **the Gridworld**. This environment is a 3×4 grid world in which the agent starts at the bottom-left corner, and it has to reach the top-right corner. There is a hole south of the goal and a wall near the start. The transition function has a 20% noise; that is, 80% the action succeeds, and 20% it fails uniformly at random in orthogonal directions. The reward function is a -0.04 living penalty, a $+1$ for landing on the goal, and a -1 for landing on the hole.

The transition function is 80% success,
10% to one side and 10% to the other

The Goal state gives +1
reward to the agent and the
episode terminates



1 - Create the environment Gridworld as described above.

YOUR CODE HERE

```
# The environment should be a class with the following methods:
# `__init__(self)`: initialize the environment
# `reset(self)`: reset the environment to the initial state
# `step(self, action)`: take an action in the environment.
# The action should be an integer between 0 and 3, where 0 means left,
# 1 means down, 2 means right, and 3 means up.
# The method should return a tuple (state, reward, done),
# where state is the new state, reward is the reward obtained, and done is
# a boolean indicating whether the episode is finished.
```

```
UP, DOWN, LEFT, RIGHT = range(4)
```

```
class GridWorld:
```

```
    # 0:UP, 1:DOWN, 2:LEFT 3:RIGHT
```

```
    def __init__(self):
        self.reset()
```

```
    def reset(self):
        self.observation_space = 12
        self.state = 8
        self.columns = 4
        self.rows = 3
        self.terminated = False
        self.reward = 0
```



```
        return self.state

def step(self, action):
    if self.terminated: raise ValueError('Episode has terminated')
    if action not in [UP, DOWN, LEFT, RIGHT]: raise ValueError('Invalid action')

    # add stochasticity
    if action == UP: action = np.random.choice([UP, LEFT, RIGHT], p=[0.8, 0.1,
        ↪ 0.1])
    if action == DOWN: action = np.random.choice([DOWN, LEFT, RIGHT], p=[0.8, 0.1,
        ↪ 0.1])
    if action == LEFT: action = np.random.choice([LEFT, UP, DOWN], p=[0.8, 0.1,
        ↪ 0.1])
    if action == RIGHT: action = np.random.choice([RIGHT, UP, DOWN], p=[0.8, 0.1,
        ↪ 0.1])

    # manage boundaries
    row = self.state // self.columns
    column = self.state % self.columns
    if action == LEFT: column = max(column - 1, 0)
    if action == DOWN: row = min(row + 1, self.rows - 1)
    if action == RIGHT: column = min(column + 1, self.columns - 1)
    if action == UP: row = max(row - 1, 0)

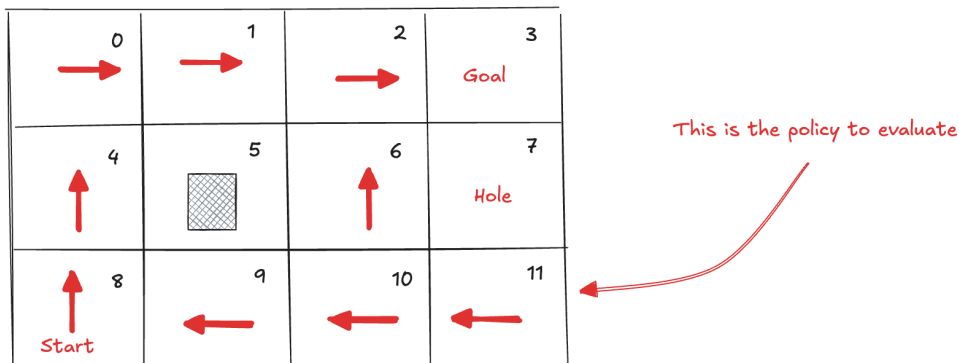
    # manage obstacles
    old_state = self.state
    self.state = row * self.columns + column
    if self.state == 5: self.state = old_state

    # manage rewards
    self.reward = -0.04
    if self.state == 3: self.terminated = True; self.reward = 1
    elif self.state == 7: self.terminated = True; self.reward = -1

    return self.state, self.reward, self.terminated, 0, 0

grid_world = GridWorld()
```

2 - Write the following policy for this environment



```
# YOUR CODE HERE
```

```
# The policy should be a function that takes an integer between 0 and 11
# (the state) and returns an integer between 0 and 3 (the action).
```

```
pi = lambda s: {
    0:RIGHT, 1:RIGHT, 2:RIGHT, 3:LEFT,
    4:UP,    5:LEFT, 6:UP,    7:LEFT,
    8:UP,    9:LEFT, 10:LEFT, 11:LEFT
}[s]
```

3 - Show the policy by printing the selected action for each state:

```
# YOUR CODE HERE
```

```
# The function should print the policy in a human-readable format.
# For example, the output could be:
# state: 0 → action: RIGHT
# state: 1 → action: RIGHT
# etc ...
```

```
def print_policy(pi, env):
    for state in range(env.observation_space):
        action = pi(state)
        action_icon = ' '
        if action == UP: action_icon = 'UP'
        if action == DOWN: action_icon = 'DOWN'
        if action == LEFT: action_icon = 'LEFT'
        if action == RIGHT: action_icon = 'RIGHT'
        print('state:', state, '→', 'action:', action_icon)
```

```
print_policy(pi, grid_world)
```

```
state: 0 -> action: RIGHT
state: 1 -> action: RIGHT
state: 2 -> action: RIGHT
state: 3 -> action: LEFT
state: 4 -> action: UP
state: 5 -> action: LEFT
state: 6 -> action: UP
state: 7 -> action: LEFT
state: 8 -> action: UP
state: 9 -> action: LEFT
state: 10 -> action: LEFT
state: 11 -> action: LEFT
```

4 - Now, evaluate this policy using TD(λ)

```
# YOUR CODE HERE

# you need to decay alpha

import numpy as np

def decay_alpha(init_value, min_value, decay_steps, max_steps):

    # calculate the number of the remaining steps after the decay
    rem_steps = max_steps - decay_steps

    # logspace returns numbers spaced evenly on a log scale
    # base^start is the starting value of the sequence,
    # base^stop is the final value of the sequence
    # num is the number of values to generate
    # base is the base of the log space
    values = np.logspace(start=0, stop=-2, num=decay_steps, base=10)

    # because the values may not end exactly at 0, given it's the log,
    # we change them to be between 0 and 1 so that the curve looks smooth and nice.
    values = (values - values.min()) / (values.max() - values.min())
```

```
# linear transformation and get points between init_value and min_value
values = (init_value - min_value) * values + min_value

# repeats the rightmost value rem_step number of times
values = np.pad(values, (0, rem_steps), 'edge')

return values
```

```
# YOUR CODE HERE

# you need to decay the discount

def decay_discounts(gamma, max_steps):
    discounts = np.logspace(start=0, stop=max_steps, num=max_steps, base=gamma,
↪ endpoint=False);
    return discounts
```

```
# YOUR CODE HERE

# you need the td_lambda implementation

def td_lambda(pi, env, gamma=1.0, lambda_=0.2,
              init_alpha=0.5, min_alpha=0.01,
              decay_episodes=350, n_episodes=500):

    # calculate all alphas at once
    alphas = decay_alpha(init_alpha, min_alpha, decay_episodes, n_episodes);

    # initialize the current estimate of the state-value function
    v = np.zeros(env.observation_space, dtype=float)

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float)

    # initialize the eligibility trace vector
    E = np.zeros(env.observation_space, dtype=float)

    # loop for every episode
    for e in range(n_episodes):

        # set E to zero every new episode
```

```
E.fill(0)

# get the initial state
state, done = env.reset(), False

# get into the time step loop
while not done:

    # sample the policy pi for the action to take in state
    action = pi(state)

    # interact with the environment for one step and get the experience tuple
    next_state, reward, terminated, truncated, info = env.step(action)
    if(terminated or truncated):
        done = True;

    # use that experience to calculate the TD error as usual
    td_target = reward + gamma * v[next_state] * (not done)
    td_error = td_target - v[state]

    # increment the eligibility of state by 1
    E[state] = E[state] + 1

    # apply the error update to all eligible states as indicated by E
    v = v + alphas[e] * td_error * E

    # decay E
    E = lambda_ * E

    # update the state variable for the next iteration
    state = next_state

    v_track[e] = v

return v, v_track
```

```
# YOUR CODE HERE

# run the algorithm and print the value function

v_td_lambda, v_td_lambda_track = td_lambda(pi, grid_world, lambda_=0.3,
↳ n_episodes=500)
```

```
print(v_td_lambda)
```

```
[0.8415656  0.89934748 0.95800912 0.          0.79107068 0.
 0.61815666 0.          0.74072074 0.60860885 0.          0.          ]
```

5 - Plot the estimated value for state 8, 6 and 2 for all episodes

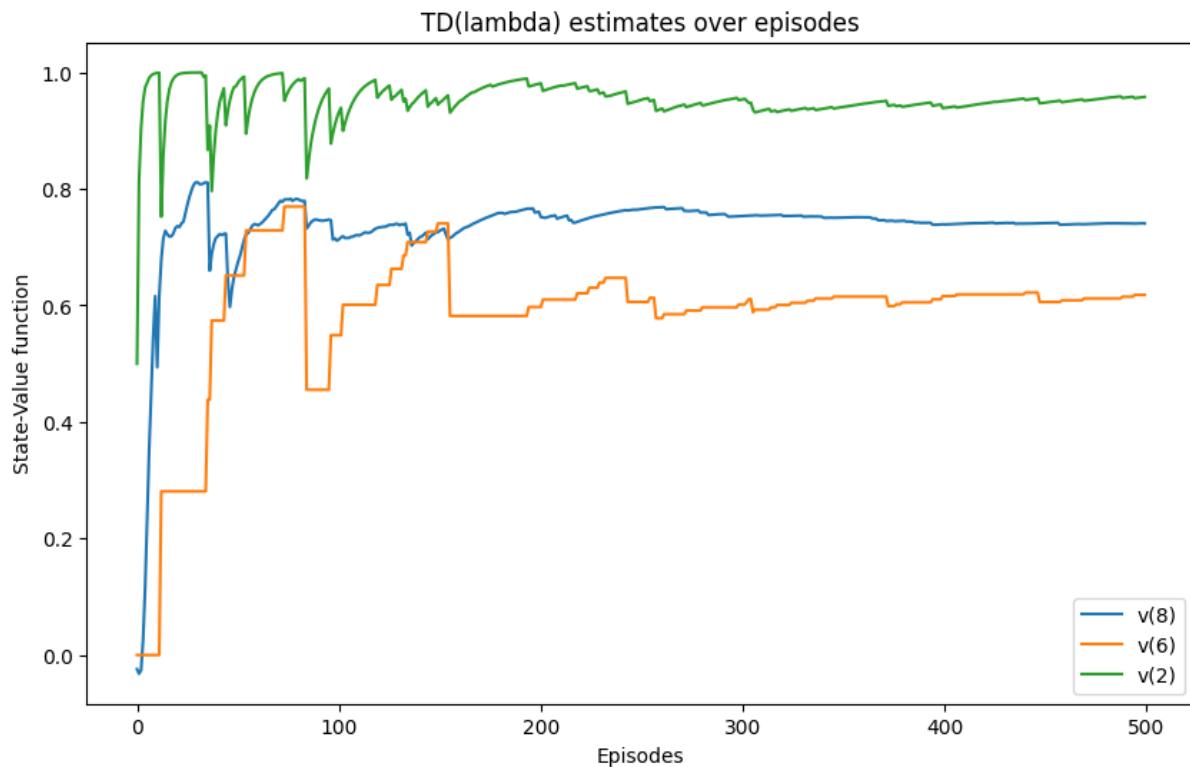
```
# YOUR CODE HERE

# plot the value function over episodes for state 8, 6 and 2

import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
legends = ['v(8)', 'v(6)', 'v(2)']
plt.plot(v_td_lambda_track[:,8])
plt.plot(v_td_lambda_track[:,6])
plt.plot(v_td_lambda_track[:,2])
plt.title('TD(lambda) estimates over episodes')
plt.ylabel('State-Value function')
plt.xlabel('Episodes')
plt.legend(legends)

plt.show()
```



3 Blackjack

The game starts with the dealer having one face up and one face down card, while the player has two face up cards. All cards are drawn from an infinite deck (i.e. with replacement). The card values are:

- Face cards (Jack, Queen, King) have a point value of 10
- Aces can either count as 11 (called a "usable ace") or 1
- Numerical cards (2-9) have a value equal to their number

The player has the sum of cards held. The player can request additional cards (hit) until they decide to stop (stick) or exceed 21 (bust, immediate loss). After the player sticks, the dealer reveals their facedown card, and draws cards until their sum is 17 or greater. If the dealer goes bust, the player wins. If neither the player nor the dealer busts, the outcome (win, lose, draw) is decided by whose sum is closer to 21.

There is two possible actions:

- 0: Stick
- 1: Hit

The observation consists of a 3-tuple containing: the player's current sum, the value of the dealer's one showing card (1-10 where 1 is ace), and whether the player holds a usable ace (0 or 1).

The player receives positive +1 rewards for winning the game, negative -1 for loosing the game and 0 reward in case of drawn.

1 - Import the 'Blackjack-v1' environment from Gymnasium:

```
# You can get the environment from Gymnasium in the same way we got 'Frozen Lake'
# in order to visually plot the environment you can import it
# using render_mode="rgb_array"

import gymnasium as gym;
blackjack = gym.make('Blackjack-v1', render_mode="rgb_array");
```

2 - Create a random policy as a baseline:

```
# You have to create a function that get in input a state and provide a random
# action (in the range [0;1])

import numpy as np

def pi_random(s):
    return np.random.choice([0,1]);
```

3 - Show the policy in action by rendering the environment several times after different decisions from the random policy:

```
# Reuse the show_policy function we implement for the lab on DP:

import time
import matplotlib.pyplot as plt
from IPython.display import clear_output

def show_policy(env, pi, n_episodes=1):
    for _ in range(n_episodes):
        state, info = env.reset()
        done = False;
        while not done:
            action = pi(state);
            state, reward, terminated, truncated, info = env.step(action);
```



```

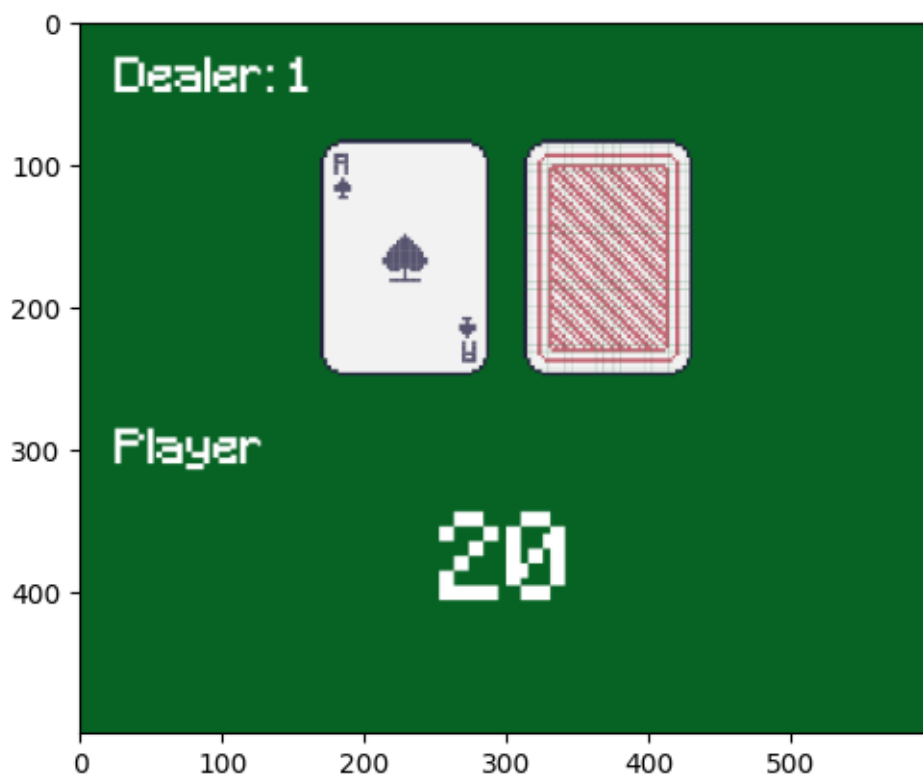
        if(terminated or truncated): done = True;
        clear_output(wait=True);
        plt.imshow(env.render());
        plt.show();

    # Show real cards from env internals
    print("Player hand:", env.unwrapped.player)
    print("Dealer hand:", env.unwrapped.dealer)

    if(reward == 1): print('You win!');
    elif(reward == -1): print('You lose!');
    else: print('Draw!');

show_policy(blackjack, pi_random);

```



```

Player hand: [10, 10]
Dealer hand: [1, 1, 4, 6, 5]
You win!

```

4 - Write a brute-force function in order to evaluate the probability of success and the average

return obtained by a policy, the evaluate the random policy:

```
# You can reuse the "evaluate" function we implment for the lab on DP,
# however pay attention on how to calculte the success rate, you need to
# consider that the agent reaches the goal if it obtains a reward of 1 at
# the end of an episode.

def evaluate(env, pi, n_episodes=10000):
    success = 0;
    returns = []
    for _ in range(n_episodes):
        done = False;
        steps = 0;
        state, info = env.reset();
        returns.append(0.0)
        while not done:
            state, reward, terminated, truncated, info = env.step(pi(state))
            if(terminated or truncated):
                done = True;
                returns[-1] += reward;
                steps += 1;
            if terminated and reward == 1:
                success += 1;
    return (success/n_episodes)*100, np.mean(returns);

probability_success, mean_return = evaluate(blackjack, pi_random);

print("Probability of success ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);
```

Probability of success 27.76 %

Obtains an average undiscounted return of -0.4002

5 - Notice that the state space is not a single integer, but a tuple of discrete states. In order to use the state as an index in the value function, you need to convert the tuple of states into a single value.

```
# To convert the tuple of discrete states
# (Discrete(32), Discrete(11), Discrete(2)) into a single index,
# you can treat the tuple as a coordinate in a 3D grid and
# compute a unique index for each combination of values.
```

```
# This is essentially flattening the multi-dimensional space
# into a one-dimensional array.
# Given a tuple of states (s1, s2, s3) where:
# - s1 is in the range 0 to 31 (32 possible values),
# - s2 is in the range 0 to 10 (11 possible values),
# - s3 is in the range 0 to 1 (2 possible values),
# You can calculate the index by assuming that s3 is the least
# significant and s1 is the most significant:

# index = s1 * (11 * 2) + s2 * 2 + s3

def state_to_index(s1, s2, s3):
    index = s1 * (11 * 2) + s2 * 2 + s3
    return index
```

6 - Use Double Q-learning to calculate the optimal policy and the optimal state-value function:

```
# You can reuse the function we apply to the Slippery Walk environment, in particular
# select_action() to use an epsilon greedy exploration strategy, decay_schedule() to
# calculate learning rate and epsilon values and double_q_learning() to implement the
# algorithm. However, you need to use the state_to_index() function to convert the
# state into an index.
```

```
def select_action(state, q, epsilon=0.1):

    # if the random number is greater than epsilon,
    if np.random.uniform() > epsilon:
        # pick the action with the highest Q value
        action = np.argmax(q[state]);
    else:
        # otherwise, pick a random action
        action = np.random.randint(len(q[0]));

    return action;
```

```
def decay_schedule(init_value, min_value, decay_steps, max_steps):

    # calculate the number of the remaining steps after the decay
    rem_steps = max_steps - decay_steps;
```

```
# logspace returns numbers spaced evenly on a log scale
# base^start is the starting value of the sequence,
# base^stop is the final value of the sequence
# num is the number of values to generate
# base is the base of the log space
values = np.logspace(start=0, stop=-2, num=decay_steps, base=10);

# because the values may not end exactly at 0, given it's the log,
# we change them to be between 0 and 1 so that the curve looks smooth and nice.
values = (values - values.min()) / (values.max() - values.min());

# linear transformation and get points between init_value and min_value
values = (init_value - min_value) * values + min_value;

# repeats the rightmost value rem_step number of times
values = np.pad(values, (0, rem_steps), 'edge');

return values;
```

```
def double_q_learning(env, gamma=0.99,
                      init_alpha=0.5, min_alpha=0.01,
                      init_epsilon=1.0, min_epsilon=0.1,
                      decay_episodes=2000, n_episodes=3000):

    # calculate the observation space size
    n_states = state_to_index(env.observation_space[0].n, env.observation_space[1].n,
    ↪ int(env.observation_space[2].n));

    # calculate the learning rate schedule for all episodes
    alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

    # calculate epsilon for all episodes
    epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

    # initialize the two action-value functions
    q1 = np.zeros((n_states, env.action_space.n), dtype=float);
    q2 = np.zeros((n_states, env.action_space.n), dtype=float);

    # the episode loop
    for e in range(n_episodes):

        # we start episode by resetting the environment and the done flag
```

```
state, done = env.reset(), False;
state = state_to_index(state[0][0], state[0][1], int(state[0][2]));

# repeat until we hit a terminal state
while not done:

    # we use the mean of our two action-value functions to select action
    action = select_action(state, (q1 + q2)/2., epsilons[e]);

    # step the environment using that action
    next_state, reward, terminated, truncated, info = env.step(action);
    next_state = state_to_index(next_state[0], next_state[1],
    ↪ int(next_state[2]));
    if(terminated or truncated):
        done = True;

    # flip a coin to determine an update to q1 or q2
    if np.random.randint(2):

        # use the action q1 thinks is best ...
        argmax_q1 = np.argmax(q1[next_state]);

        # ...but get the value from q2 to calculate the target
        dq_learning_target = reward + gamma * q2[next_state][argmax_q1];

        # calculate the error
        dq_learning_error = dq_learning_target - q1[state][action];

        # update q1
        q1[state][action] = q1[state][action] + alphas[e] * dq_learning_error;

    else:

        # use the action q2 thinks is best ...
        argmax_q2 = np.argmax(q2[next_state]);

        # ...but get the value from q1 to calculate the target
        dq_learning_target = reward + gamma * q1[next_state][argmax_q2];
        dq_learning_error = dq_learning_target - q2[state][action];

        # update q2
        q2[state][action] = q2[state][action] + alphas[e] * dq_learning_error;
```

```
        # update state
        state = next_state;

    # extract the value functions
    q = (q1 + q2)/2.;
    v = np.max(q, axis=1);

    # and the greedy policy
    def pi(s):
        return np.argmax(q[state_to_index(s[0], s[1], int(s[2]))]);

    return q, v, pi;
```

```
q, v, pi = double_q_learning(blackjack);
```

7 - Calculate the performance of the obtained optimal policy using the brute force approach:

```
# You can reuse the "evaluation" function written before,
# in order to evaluate the optimal policy

probability_success, mean_return = evaluate(blackjack, pi);
print("Probability of success ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);
```

```
Probability of success  40.160000000000004 %
Obtains an average undiscounted return of  -0.1124
```

8 - Show the optimal policy:

```
# You can reuse the "show_policy" function written before,
# in order to show the optimal policy

show_policy(blackjack, pi, n_episodes=1)
```



Player hand: [3, 10, 6]

Dealer hand: [7, 5, 3, 8]

You win!

4 Frozen Lake

Apply and compare SARSA(λ), Q(λ), DynaQ and Trajectory sampling agents on the Frozen Lake (FL) environment using a discount factor $\gamma=0.99$. For the comparison, you can plot only states (0, 6, 10). Get also the optimal state-value functions using Dynamic Programming.

Remember: FL is a simple grid-world environment. It has 16 states and 4 actions. The goal of the agent is to go from a start location to a goal location while avoiding falling into holes. All transitions landing on the goal state provide a +1 reward, while every other transition in the entire grid world provides no reward. The challenge is that the surface of the lake is frozen, and therefore slippery. So actions have stochastic effects, and the agent moves only a third of the time as intended. The other two-thirds is split evenly in orthogonal directions.

FL is a more challenging environment than Slippery Walk environment. Therefore, one of the most important changes you need to make is to increase the number of episodes the agent interacts with the environment. While in the Slippery Walk environment we allow the agent to interact for only 3.000 episodes, in the FL environment, let your agent gather experience for 10.000 episodes.

1 - Import the FrozenLake environment and extract it MPD dynamics.

```
# YOUR CODE HERE

# You can get the environment from Gymnasium 'FrozenLake-v1';
# In order to visually plot the environment you can import it
# using render_mode="rgb_array"
# You can also import the actions LEFT, DOWN, RIGHT, UP from gym
# Finally, you can import the transition probabilities P from gym

import gymnasium as gym

LEFT, DOWN, RIGHT, UP = range(4)
frozen_lake = gym.make('FrozenLake-v1', render_mode="rgb_array")
```

2- Create a random policy as a baseline

```
# YOUR CODE HERE

# You have to create a function that get in input a state and provide a random action

import numpy as np

def pi_random(s):
    return np.random.choice(4);
```

3 - Show the policy in action by rendering the environment several times after different decisions from the random policy:

```
# YOUR CODE HERE

# You have to use a while loop in order to provide the current state to
# the policy and then make a step in the environment using the action
# provided by the policy.
# Try to create a function (to be called "show_policy")
```

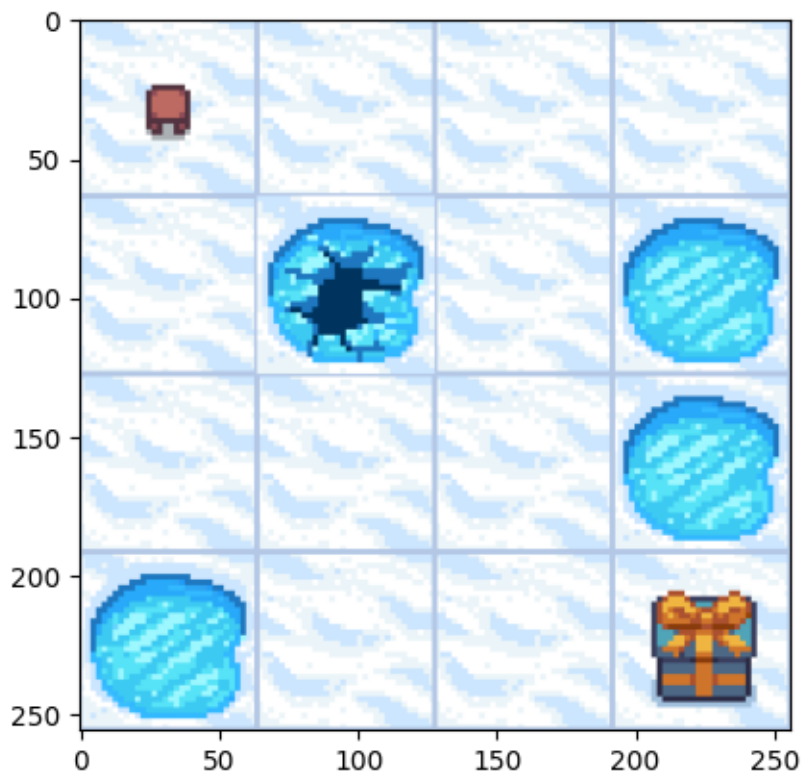


```
# to be reused with other policies later.
# Hints: env.render() provides you an array representing
# an image of the environment; plt.imshow() can be used to visualize
# the image on the screen.

import time
import matplotlib.pyplot as plt
from IPython.display import clear_output

def show_policy(env, pi, n_episodes=1, max_steps=50):
    for _ in range(n_episodes):
        state, _ = env.reset()
        done = False
        steps = 0;
        while not done and steps < max_steps:
            action = pi(state);
            state, reward, done, _, _ = env.step(action)
            clear_output(wait=True)
            plt.imshow(env.render())
            plt.show()
            steps += 1
            print('Step :', steps, 'Action: ', action)
            time.sleep(0.5)

show_policy(frozen_lake, pi_random)
```



Step : 6 Action: 3

4 - Solve the environment using Dynamic Programming:

```
# YOUR CODE HERE

# Copy the value iteration algorithm from the lecture notes.

def value_iteration(env, gamma=1.0, theta=1e-10):

    # we need to unwrap the environment to access its
    # internal dynamic
    dynamics = env.unwrapped.P;

    # initialize a state-value function
    v = np.zeros(len(dynamics))

    while True:
        q = np.zeros((len(dynamics), len(dynamics[0])))
```

```

# for every transition of every action in every state, we ...
for s in range(len(dynamics)):
    for a in range(len(dynamics[s])):
        for prob, next_state, reward, done in dynamics[s][a]:
            # ... calculate the action-value function
            q[s][a] += prob * (reward + gamma * v[next_state])

# After each sweep over the state space, make sure the state-value function
# keeps changing,
# otherwise, we found the optimal state-value function and should break out
if np.max(np.abs(v - np.max(q, axis=1))) < theta:
    break;

# we don't need a separate policy-improvement phase
v = np.max(q, axis=1)

# only at the end, we extract the optimal policy
def pi(s):
    return np.argmax(q[s])

return v, pi

```

```

# YOUR CODE HERE

# Solve the problem using the algorithm and exploiting the MDP dynamics

optimal_v, optimal_pi = value_iteration(frozen_lake, gamma=0.99)

```

5 - Print the values of the optimal state-value function

```

# YOUR CODE HERE

# Copy the print_state_value_function function from the lecture notes and use it to
# print the state-value function for the best policy

def print_state_value_function(v, env):
    dynamics = env.unwrapped.P;
    n_cols = 4;
    print("State-value function:");
    for s in range(len(dynamics)):
        if np.all([done for action in dynamics[s].values() for _, _, _, done in
                    ↪ action]):

```

```

        print(" X ", end="\t");
    else:
        print(np.round(v[s], 2), end="\t");
    if (s+1)%n_cols==0:
        print(" ");
print_state_value_function(optimal_v, frozen_lake)

```

State-value function:

```

0.54 0.5 0.47 0.46
0.56 X 0.36 X
0.59 0.64 0.62 X
X 0.74 0.86 X

```

6 - Implement the SARSA(lambda) algorithm and use it against the FL environment

```

# YOUR CODE HERE

# Copy the decay_schedule function from the lecture notes

def decay_schedule(init_value, min_value, decay_steps, max_steps):

    # calculate the number of the remaining steps after the decay
    rem_steps = max_steps - decay_steps;

    # logspace returns numbers spaced evenly on a log scale
    # base^start is the starting value of the sequence,
    # base^stop is the final value of the sequence
    # num is the number of values to generate
    # base is the base of the log space
    values = np.logspace(start=0, stop=-2, num=decay_steps, base=10);

    # because the values may not end exactly at 0, given it's the log,
    # we change them to be between 0 and 1 so that the curve looks smooth and nice.
    values = (values - values.min()) / (values.max() - values.min());

    # linear transformation and get points between init_value and min_value
    values = (init_value - min_value) * values + min_value;

    # repeats the rightmost value rem_step number of times
    values = np.pad(values, (0, rem_steps), 'edge');

```

```
return values;
```

```
# YOUR CODE HERE
```

```
# Copy the decay_discounts function from the lecture notes
```

```
def decay_discounts(gamma, max_steps):
    discounts = np.logspace(start=0, stop=max_steps, num=max_steps, base=gamma,
↪ endpoint=False);
    return discounts
```

```
# YOUR CODE HERE
```

```
# Copy the select_action function from the lecture notes
```

```
def select_action(state, q, epsilon=0.1):

    # if the random number is greater than epsilon,
    if np.random.uniform() > epsilon:
        # pick the action with the highest Q value
        action = np.argmax(q[state]);
    else:
        # otherwise, pick a random action
        action = np.random.randint(len(q[0]));

    return action;
```

```
# YOUR CODE HERE
```

```
# Copy the sarsa_lambda function from the lecture notes and modify it in order to ask
↪ the environment the number
# of states and actions following the Gymnasium interface (.n), and extract the id of
↪ the starting state from the first
# output of the reset() function.
```

```
def sarsa_lambda(env, gamma=0.99,
                 init_alpha=0.5, min_alpha=0.01,
                 init_epsilon=1.0, min_epsilon=0.1,
                 lambda_=0.5, replacing_traces=True,
                 decay_episodes=2500, n_episodes=10000):
```

```
# calculate the learning rate schedule for all episodes
alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

# calculate epsilon for all episodes
epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

# initialize the current estimate of the action-value function
q = np.zeros((env.observation_space.n, env.action_space.n), dtype=float);

# eligibility traces
E = np.zeros((env.observation_space.n, env.action_space.n), dtype=float);

# create a list to save copies of v for offline analysis
v_track = np.zeros((n_episodes, env.observation_space.n), dtype=float);

# the episode loop
for e in range(n_episodes):

    # every new episode, set the eligibility of every state to zero
    E.fill(0);

    # # we start episode by resetting the environment and the done flag
    state, done = env.reset()[0], False;

    # select the action (perhaps exploratory) for the initial state
    action = select_action(state, q, epsilons[e]);

    # repeat until we hit a terminal state
    while not done:

        # step the environment using that action
        next_state, reward, terminated, truncated, info = env.step(action);
        if(terminated or truncated):
            done = True;

        # obtain the action for the next step
        next_action = select_action(next_state, q, epsilons[e]);

        # calculate the target using that next state-action pair
        sarsa_lambda_target = reward + gamma * q[next_state][next_action];

        # calculate the error
```

```

sarsa_lambda_error = sarsa_lambda_target - q[state][action];

# increment the state-action pair trace,
# and clip it to 1 if it's a replacing trace
if replacing_traces:
    E[state].fill(0);
E[state][action] = E[state][action] + 1;
if replacing_traces:
    E.clip(0, 1, out=E);

# apply the error to all eligible state-action pairs at once
# even though we're using the entire q-table, E will be mostly 0,
# and greater than zero for eligible pairs
q = q + alphas[e] * sarsa_lambda_error * E;

# decay the eligibility matrix
E = lambda_ * E;

# update state and action for the next step
state, action = next_state, next_action;

# extract the state-value function
v = np.max(q, axis=1);

# and the greedy policy
def pi(s):
    return np.argmax(q[s]);

# save values for post analysis
v_track[e] = v;

return q, v, pi, v_track;

```

```
# YOUR CODE HERE
```

```
# Apply sarsa(lambda) for 10.000 episodes and gamma = 0.99
```

```
q_sarsa_lambda, v_sarsa_lambda, pi_sarsa_lambda, v_track_sarsa_lambda =
    sarsa_lambda(frozen_lake);
```

7 - Plot the state-value functions for states (0, 6, 10) for SARSA(lambda) and compare with the optimal values. Then observe how the agent struggle to estimate the optimal state-value

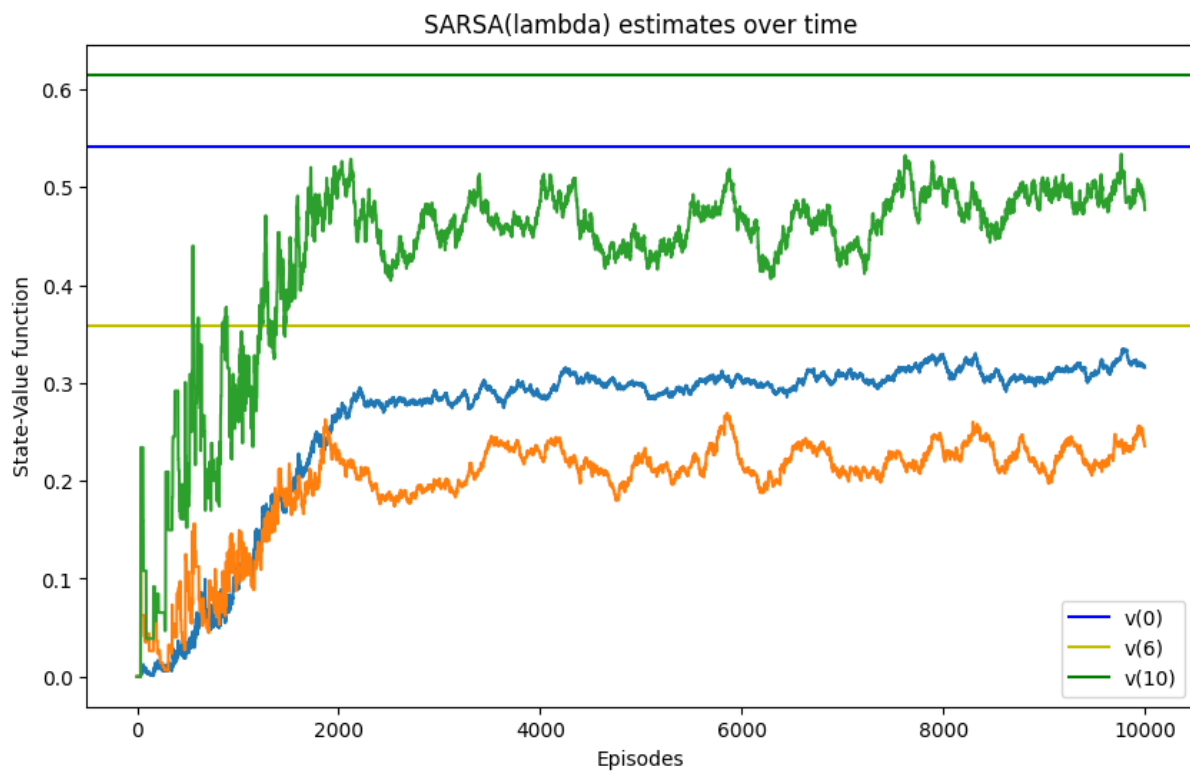
function.

```
# YOUR CODE HERE

# import matplotlib.pyplot as plt and plot the V_track_sarsa_lambda for each episode
↳ for states 0, 6 and 10
# draw a horizontal line for the optimal_V for states 0, 6 and 10

import matplotlib.pyplot as plt

legends = ['v(0)', 'v(6)', 'v(10)'];
plt.figure(figsize=(10,6));
plt.title('SARSA(lambda) estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.axhline(y=optimal_v[0], color='b', linestyle='-');
plt.axhline(y=optimal_v[6], color='y', linestyle='-');
plt.axhline(y=optimal_v[10], color='g', linestyle='-');
plt.plot(v_track_sarsa_lambda[:,[0,6,10]]);
plt.legend(legends);
plt.show();
```



8 - Implement the Q(λ) algorithm and use it against the FL environment

```
# YOUR CODE HERE

# Copy the q_lambda function from the lecture notes and modify it in order to ask the
↳ environment the number
# of states and actions following the Gymnasium interface (.n) and extract the id of
↳ the starting state from the first
# output of the reset() function.

def q_lambda(env, gamma=0.99,
             init_alpha=0.5, min_alpha=0.01,
             init_epsilon=1.0, min_epsilon=0.1,
             lambda_=0.5, replacing_traces=True,
             decay_episodes=2500, n_episodes=10000):

    # calculate the learning rate schedule for all episodes
    alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

    # calculate epsilon for all episodes
    epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

    # initialize the current estimate of the action-value function
    q = np.zeros((env.observation_space.n, env.action_space.n), dtype=float);

    # eligibility traces
    E = np.zeros((env.observation_space.n, env.action_space.n), dtype=float);

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space.n), dtype=float);

    # the episode loop
    for e in range(n_episodes):

        # every new episode, set the eligibility of every state to zero
        E.fill(0);

        # we start episode by resetting the environment and the done flag
        state, done = env.reset()[0], False;

        # notice: we are preselecting the action as in SARSA, (we didn't do that in
        ↳ Q-learning)
        # this is because we need to check whether our next action is greedy for
        ↳ deciding
```

```
# to decay the eligibility trace
action = select_action(state, q, epsilons[e])

# repeat until we hit a terminal state
while not done:

    # step the environment using that action
    next_state, reward, terminated, truncated, info = env.step(action);
    if(terminated or truncated):
        done = True;

    # select the next_action SARSA-style
    next_action = select_action(next_state, q, epsilons[e])

    # use it to verify that the action on the next step
    # will still come from the greedy policy
    next_action_is_greedy = q[next_state][next_action] == q[next_state].max()

    # still calculate the target as in regular Q-learning, using the max
    q_lambda_learning_target = reward + gamma * q[next_state].max();

    # and use the target to calculate TD error
    q_lambda_learning_error = q_lambda_learning_target - q[state][action];

    # increment the state-action pair trace,
    # and clip it to 1 if it's a replacing trace
    if replacing_traces:
        E[state].fill(0);
    E[state][action] = E[state][action] + 1
    if replacing_traces:
        E.clip(0, 1, out=E);

    # as before, we multiply the entire eligibility trace matrix
    # by the error and the learning rate corresponding to episode e,
    q = q + alphas[e] * q_lambda_learning_error * E

    # if the action we'll take on the next state (which we already selected)
    # is a greedy action,
    # then we decay the eligibility matrix as usual, otherwise, we must reset
    # the eligibility matrix to zero because we'll no longer be learning about
    # the greedy policy
    if next_action_is_greedy:
        E = lambda_ * E;
```

```

    else:
        E.fill(0);

        # update state and action for the next step
        state, action = next_state, next_action

        # extract the state-value function
        v = np.max(q, axis=1);

        # and the greedy policy
        def pi(s):
            return np.argmax(q[s]);

        # save values for post analysis
        v_track[e] = v;

    return q, v, pi, v_track;

```

```

# YOUR CODE HERE

# Apply q(lambda) for 10.000 episodes and gamma = 0.99

q_ql_lambda, v_ql_lambda, pi_ql_lambda, v_track_ql_lambda = q_lambda(frozen_lake);

```

9- Plot the state-value functions for states (0, 6, 10) for $Q(\lambda)$ and compare with the optimal values. Then observe as $Q(\lambda)$ agent is moving the estimates of the optimal state-value function toward the true values, unlike SARSA(λ). Now, to be clear, this is a matter of the number of steps, also SARSA(λ) would converge to the true values if given more episodes.

```

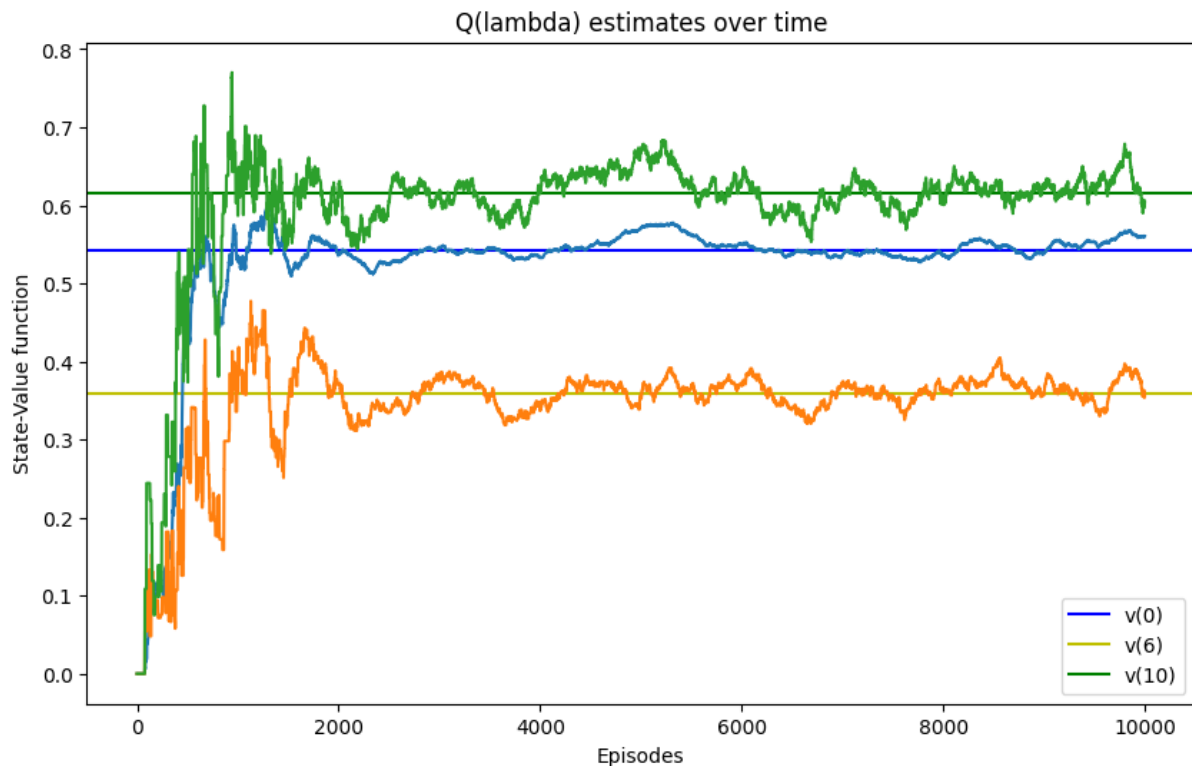
# YOUR CODE HERE

# import matplotlib.pyplot as plt and plot the V_track_q_lambda for each episode for
↪ states 0, 6 and 10
# draw a horizontal line for the optimal_V for states 0, 6 and 10

plt.figure(figsize=(10,6));
plt.title('Q(lambda) estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.axhline(y=optimal_v[0], color='b', linestyle='--');

```

```
plt.axhline(y=optimal_v[6], color='y', linestyle='-');
plt.axhline(y=optimal_v[10], color='g', linestyle='-');
plt.plot(v_track_ql_lambda[:,[0,6,10]]);
plt.legend(legends);
plt.show();
```



10 - Implement the DynaQ algorithm and use it against the FL environment.

```
# YOUR CODE HERE

# Copy the dyna_q function from the lecture notes and modify it in order to ask the
# environment the number
# of states and actions following the Gymnasium interface (.n) and extract the id of
# the starting state from the first
# output of the reset() function.

def dyna_q(env, gamma=0.99,
           init_alpha=0.5, min_alpha=0.01,
           init_epsilon=1.0, min_epsilon=0.1,
           n_planning=3,
           decay_episodes=2500, n_episodes=10000):
```

```
# calculate the learning rate schedule for all episodes
alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

# calculate epsilon for all episodes
epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

# initialize the current estimate of the action-value function
q = np.zeros((env.observation_space.n, env.action_space.n), dtype=float);

# create a list to save copies of v for offline analysis
v_track = np.zeros((n_episodes, env.observation_space.n), dtype=float);

# create a function to keep track of the transition function
# and another one to keep track of the reward signal.
transitions = np.zeros((env.observation_space.n, env.action_space.n,
↪ env.observation_space.n), dtype=int);
rewards = np.zeros((env.observation_space.n, env.action_space.n,
↪ env.observation_space.n), dtype=float);

# create a list to save copies of the model and of the planning
# for offline analysis
transitions_track = [];
planning_track = [];

# the episode loop
for e in range(n_episodes):

    # we start episode by resetting the environment and the done flag
    state, done = env.reset()[0], False;

    # repeat until we hit a terminal state
    while not done:

        # select the action, as in Q-learning
        action = select_action(state, q, epsilons[e]);

        # step the environment using that action
        next_state, reward, terminated, truncated, info = env.step(action);
        if(terminated or truncated):
            done = True;

    # start learning the model
```

```

# increment the transition count for the state-action-next_state triplet
transitions[state][action][next_state] += 1;

# calculate an incremental mean of the reward signal
# and use it to learn the reward signal
difference = reward - rewards[state][action][next_state];
rewards[state][action][next_state] += (difference /
↳ transitions[state][action][next_state]);

# calculate the target as usual in Q-learning!
# (off-policy, using the max)
dyna_target = reward + gamma * q[next_state].max();

# calculate the error
dyna_error = dyna_target - q[state][action];

# update the action-value function
q[state][action] = q[state][action] + alphas[e] * dyna_error;

# get into the planning steps, first back up the next state variable
backup_next_state = next_state;

# planning loop
for _ in range(n_planning):

    # make sure there have been updates to the Q-function before,
    # otherwise, there's not much to plan
    if q.sum() == 0:
        break;

    # select a state from a list of states already visited by
    # the agent in experience
    visited_states = np.where(np.sum(transitions, axis=(1, 2)) > 0)[0];
    state = np.random.choice(visited_states);

    # then select an action that has been taken on that state
    actions_taken = np.where(np.sum(transitions[state], axis=1) > 0)[0];
    action = np.random.choice(actions_taken);

    # use the transitions to calculate probabilities of a next state
    # and then a next state
    probs = transitions[state][action]/transitions[state][action].sum();
    next_state = np.random.choice(np.arange(env.observation_space.n),
↳ size=1, p=probs)[0];

```

```

        # use the rewards model as the reward
        reward = rewards[state][action][next_state];

        # save the planning decision for offline analysis
        planning_track.append((state, action, reward, next_state));

        # update the Q-function using that simulated experience!
        dyna_target = reward + gamma * q[next_state].max();
        dyna_error = dyna_target - q[state][action];
        q[state][action] = q[state][action] + alphas[e] * dyna_error;

        # at the end of the planning, set the state as the next state
        state = backup_next_state;

    # extract the state-value function
    v = np.max(q, axis=1);

    # and the greedy policy
    def pi(s):
        return np.argmax(q[s]);

    # save the value functions and the transitions model for analysis
    v_track[e] = v;
    transitions_track.append(transitions.copy());

    return q, v, pi, v_track, transitions_track, np.array(planning_track);

```

```
# YOUR CODE HERE
```

```
# Apply DynaQ for 10.000 episodes and gamma = 0.99
```

```
q_dyna, v_dyna, pi_dyna, v_track_dyna, transitions_track_dyna, planning_dyna =
↳ dyna_q(frozen_lake)
```

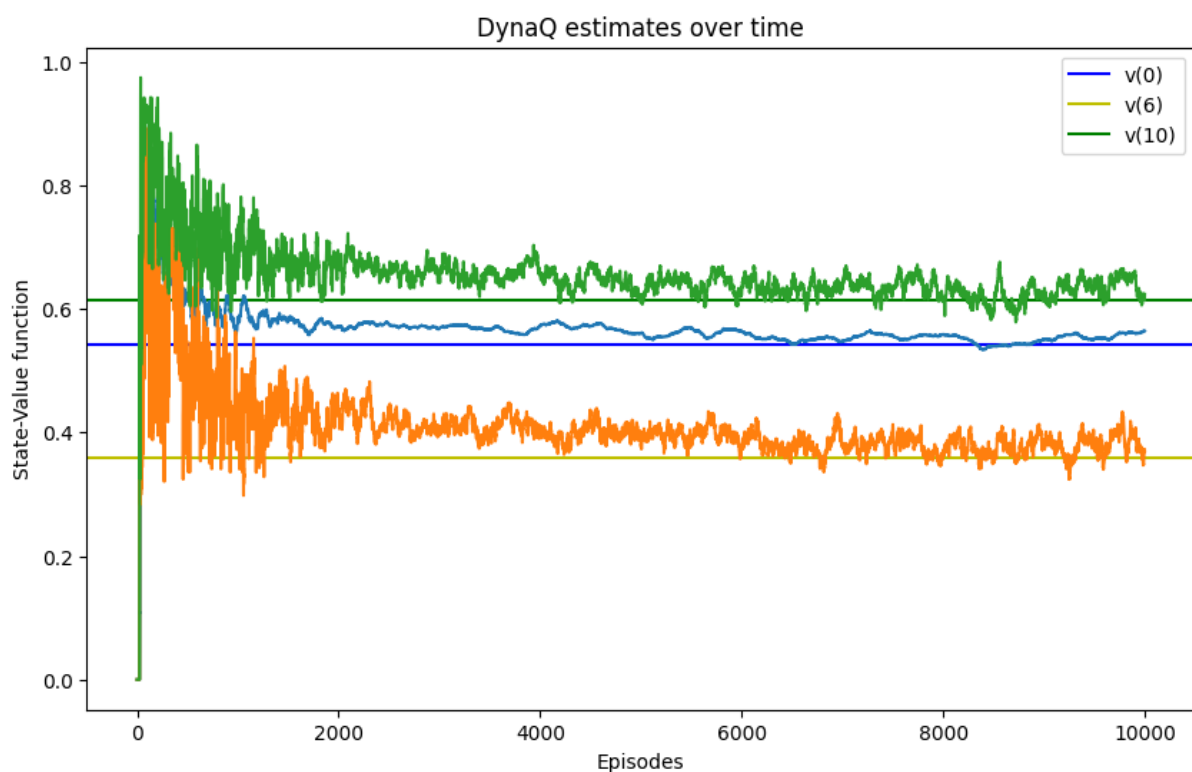
11 - Plot the state-value functions for states (0, 6, 10) for DynaQ and compare with the optimal values.

```
# YOUR CODE HERE
```

```
# import matplotlib.pyplot as plt and plot the V_track_dq for each episode for states
↳ 0, 6 and 10
```

```
# draw a horizontal line for the optimal_V for states 0, 6 and 10
```

```
plt.figure(figsize=(10,6));
plt.title('DynaQ estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.axhline(y=optimal_v[0], color='b', linestyle='-');
plt.axhline(y=optimal_v[6], color='y', linestyle='-');
plt.axhline(y=optimal_v[10], color='g', linestyle='-');
plt.plot(v_track_dyna[:,[0,6,10]]);
plt.legend(legends);
plt.show();
```



12 - Implement the Trajectory Sampling algorithm and use it against the FL environment.

```
# YOUR CODE HERE
```

```
# Copy the trajectory_sampling function from the lecture notes and modify it in order
↳ to ask the environment the number
# of states and actions following the Gymnasium interface (.n) and extract the id of
↳ the starting state from the first
```



```
# output of the reset() function.

def trajectory_sampling(env, gamma=0.99,
                       init_alpha=0.5, min_alpha=0.01,
                       init_epsilon=1.0, min_epsilon=0.1,
                       n_planning=3,
                       decay_episodes=2500, n_episodes=10000):

    # calculate the learning rate schedule for all episodes
    alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

    # calculate epsilon for all episodes
    epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

    # initialize the current estimate of the action-value function
    q = np.zeros((env.observation_space.n, env.action_space.n), dtype=float);

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space.n), dtype=float);

    # create a function to keep track of the transition function
    # and another one to keep track of the reward signal.
    transitions = np.zeros((env.observation_space.n, env.action_space.n,
↪ env.observation_space.n), dtype=int);
    rewards = np.zeros((env.observation_space.n, env.action_space.n,
↪ env.observation_space.n), dtype=float);

    # create a list to save copies of the model and of the planning
    # for offline analysis
    transitions_track = [];
    planning_track = [];

    # the episode loop
    for e in range(n_episodes):

        # we start episode by resetting the environment and the done flag
        state, done = env.reset()[0], False;

        # repeat until we hit a terminal state
        while not done:

            # select the action, as in Q-learning
            action = select_action(state, q, epsilons[e])
```

```
# step the environment using that action
next_state, reward, terminated, truncated, info = env.step(action);
if(terminated or truncated):
    done = True;

# start learning the model
# increment the transition count for the state-action-next_state triplet
transitions[state][action][next_state] += 1;

# calculate an incremental mean of the reward signal
# and use it to learn the reward signal
difference = reward - rewards[state][action][next_state];
rewards[state][action][next_state] += (difference /
↳ transitions[state][action][next_state]);

# calculate the target as usual in Q-learning!
# (off-policy, using the max)
ts_target = reward + gamma * q[next_state].max();

# calculate the error
ts_error = ts_target - q[state][action];

# update the action-value function
q[state][action] = q[state][action] + alphas[e] * ts_error;

# get into the planning steps, first back up the next state variable
backup_next_state = next_state;

# planning loop
for _ in range(n_planning):

    # make sure there have been updates to the Q-function before,
    # otherwise, there's not much to plan
    if q.sum() == 0:
        break;

    # select the action following the behavioral policy or from the greedy
    ↳ policy
    # action = select_action(state, q, epsilons[e]);
    action = q[state].argmax();

    # if we haven't experienced the transition,
```

```
# planning would be a mess, so break out
if not transitions[state][action].sum():
    break;

# otherwise, get the probabilities of next_state and sample
# the model accordingly, as in DynaQ
probs = transitions[state][action]/transitions[state][action].sum();
next_state = np.random.choice(np.arange(env.observation_space.n),
↪ size=1, p=probs)[0];

# get the reward as prescribed by the reward-signal model, as in DynaQ
reward = rewards[state][action][next_state];

# save the planning decision for offline analysis
planning_track.append((state, action, reward, next_state));

# update the Q-function using that simulated experience!
ts_target = reward + gamma * q[next_state].max();
ts_error = ts_target - q[state][action];
q[state][action] = q[state][action] + alphas[e] * ts_error;

# we update the state variable before we loop and
# continue the planning steps
state = next_state;

# at the end of the planning, set the state as the next state
state = backup_next_state;

# extract the state-value function
v = np.max(q, axis=1);

# and the greedy policy
def pi(s):
    return np.argmax(q[s]);

# save the value functions and the transitions model for analysis
v_track[e] = v;
transitions_track.append(transitions.copy());

return q, v, pi, v_track, transitions_track, np.array(planning_track);
```

```
# YOUR CODE HERE

# Apply trajectory_sampling for 10.000 episodes and gamma = 0.99

s, v_ts, pi_ts, v_track_ts, transitions_track_ts, planning_ts =
    trajectory_sampling(frozen_lake);
```

13 - Plot the state-value functions for states (0, 6, 10) for Trajectory Sampling and compare with the optimal values.

```
# YOUR CODE HERE

# import matplotlib.pyplot as plt and plot the V_track_ts for each episode for states
    ↪ 0, 6 and 10
# draw a horizontal line for the optimal_V for states 0, 6 and 10

plt.figure(figsize=(10,6));
plt.title('Trajectory Sampling estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.axhline(y=optimal_v[0], color='b', linestyle='-');
plt.axhline(y=optimal_v[6], color='y', linestyle='-');
plt.axhline(y=optimal_v[10], color='g', linestyle='-');
plt.plot(v_track_ts[:,[0,6,10]]);
plt.legend(legends);
plt.show();
```



14 - Plot the state-value function mean absolute error for all the methods.

```
# YOUR CODE HERE
```

```
# Copy the moving average function from the lecture notes
```

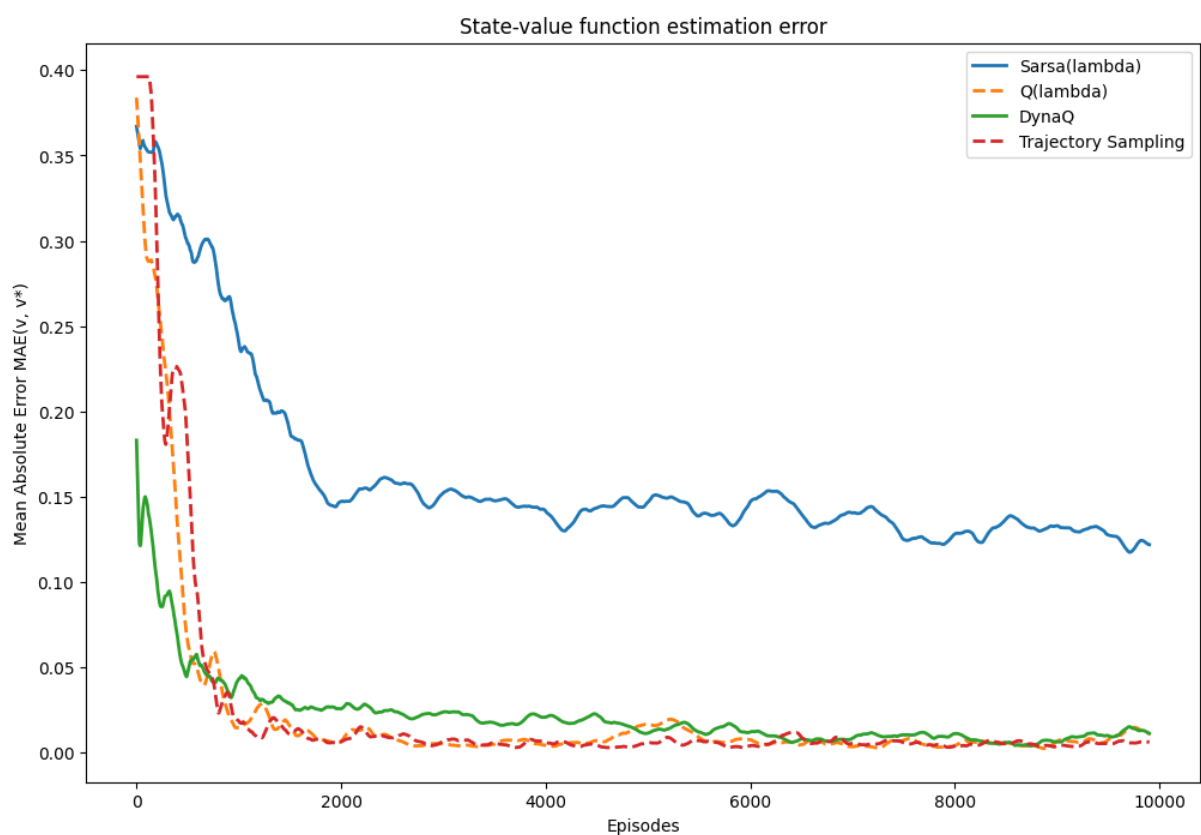
```
def moving_average(a, n=100) :
    ret = np.cumsum(a, dtype=float)
    ret[n:] = ret[n:] - ret[:-n]
    return ret[n - 1:] / n
```

```
# YOUR CODE HERE
```

```
# plot the moving average of the absolute error between the optimal_v
# and the v for each algorithm
```

```
plt.figure(figsize=(12,8));
plt.plot(moving_average(np.mean(np.abs(v_track_sarsa_lambda - optimal_v), axis=1)),
    '-', linewidth=2, label='Sarsa(lambda)');
plt.plot(moving_average(np.mean(np.abs(v_track_ql_lambda - optimal_v), axis=1)),
    '--', linewidth=2, label='Q(lambda)');
```

```
plt.plot(moving_average(np.mean(np.abs(v_track_dyna - optimal_v), axis=1)), '--',
        linewidth=2, label='DynaQ');
plt.plot(moving_average(np.mean(np.abs(v_track_ts - optimal_v), axis=1)), '--',
        linewidth=2, label='Trajectory Sampling');
plt.legend(loc=1, ncol=1);
plt.title('State-value function estimation error');
plt.xlabel('Episodes');
plt.ylabel('Mean Absolute Error MAE(v, v*)');
plt.show();
```

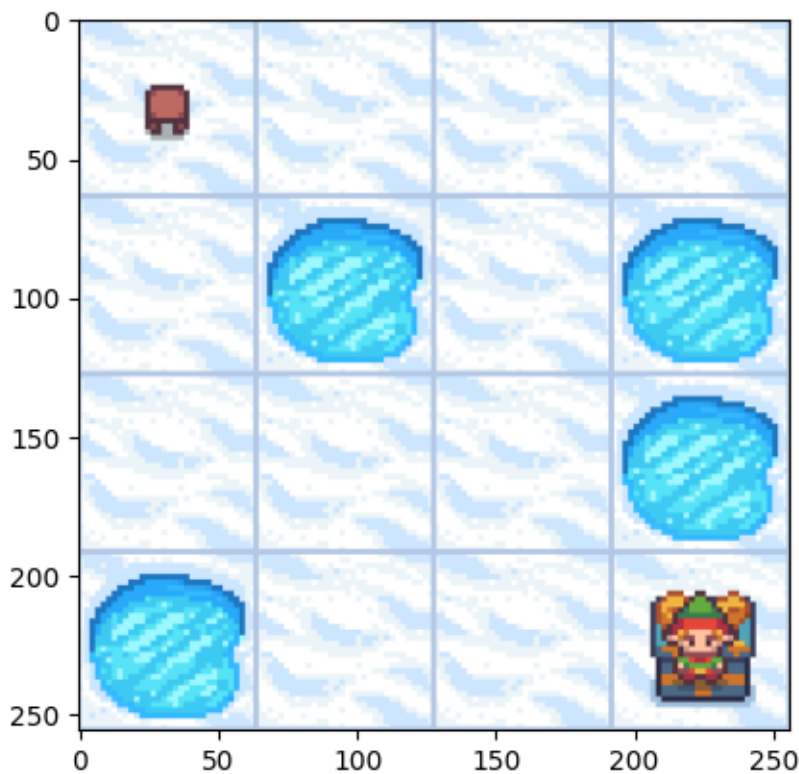


15 - Show the optimal policy:

```
# YOUR CODE HERE

# You can reuse the "show_policy" function written before,
# in order to show the optimal policy

show_policy(frozen_lake, pi_ql_lambda, n_episodes=1, max_steps=5000)
```



Step : 74 Action: 1

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click [here](https://aka.ms/vscodeJupyterKernelCrash) for more info.

View Jupyter [log](command:jupyter.viewOutput) for further details.

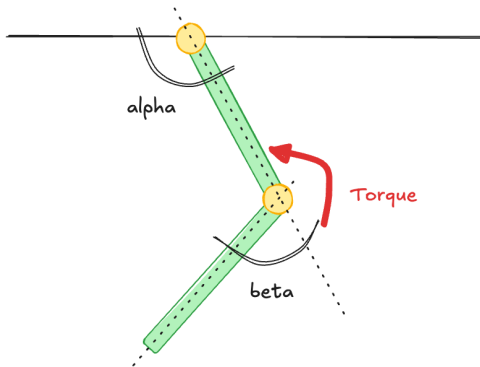
4.1 Acrobot

The Acrobot is a classic problem in control theory. The environment consists of two links connected linearly to form a chain, with one end of the chain fixed. The joint between the two links is actuated. The goal is to apply torques on the actuated joint to swing the free end of the linear chain above a given height while starting from the initial state of hanging downwards.

- The action space is discrete, deterministic, and represents the torque applied on the actuated joint between the two links: (0) apply -1 torque, (1) apply 0 torque to the actuated

joint, (2) apply 1 torque to the actuated joint

- The observation space is a 6-dimensional space with the sine and cosine of the two joint angles and their angular velocities.



The goal is to have the free-end reach a designated target height in as few steps as possible, and as such all steps that do not reach the goal incur a reward of -1. Achieving the target height results in termination with a reward of 0. The reward threshold is -100.

1 - import the environment from Gymnasium and show it with a random policy

```
# YOUR CODE HERE

# You can get the environment from Gymnasium 'Acrobot-v1';
# In order to visually plot the environment you can import it
# using render_mode="rgb_array". Get also the state size
# and action_size.

import gymnasium

env = gymnasium.make('Acrobot-v1', render_mode="rgb_array");

state_size = env.observation_space.shape;
action_size = env.action_space.n;

print('State Space: ', state_size);
print('Action Space: ', action_size);
```

State Space: (6,)

Action Space: 3

2 - Show the policy in action by rendering the environment several times after different decisions from a random policy:


```
# YOUR CODE HERE

# You can use the same function already used for the cartpole environment to
# implement a random agent and to show the policy.

import numpy as np

def random_pi(state):
    # selects an action uniformly at random
    # from the environment's action space.
    return env.action_space.sample()
```

```
import matplotlib.pyplot as plt
from IPython.display import clear_output

def show_policy(pi, n_episodes=1, max_steps=500):

    # start the episodes
    for episode in range(n_episodes):

        # reset the environment
        state, _ = env.reset();
        done = False;

        # initialize the step counter
        step = 0;

        # run the episode until done or max_steps
        while not done and step < max_steps:

            # make a step
            action = pi(state);
            state, _, terminated, truncated, _ = env.step(action);
            done = terminated or truncated;

            # render the environment
            clear_output(wait=True)
            plt.imshow(env.render())
            plt.axis("off")
            plt.title(f"Episode {episode+1}, Step {step}")
            plt.show()
            plt.close()
```

```
# increment the step counter  
step += 1;
```

```
show_policy(random_pi, n_episodes=1, max_steps=500);
```

Episode 1, Step 499



3 - Create the evaluation function for a policy

```
# YOUR CODE HERE  
  
# It is the code to run the policy several times and show the  
# average reward.  
  
def evaluate(pi, episodes=1):  
  
    # collect total rewards per episode  
    rewards = []  
  
    # loop over episodes  
    for episode in range(episodes):
```

```
# reset the environment
state, _ = env.reset();
done = False;
total_reward = 0.0;

# run an episode
while not done:
    action = pi(state);
    state, reward, terminal, truncated, _ = env.step(action)
    total_reward += reward;
    done = terminal or truncated;

# store the total reward
rewards.append(total_reward);

# return the average reward over the episodes
return float(np.mean(rewards))
```

4 - Create a neural network as a function approximator for the Q-function

```
# YOUR CODE HERE

# Add the code for a neural network using the PyTorch library and
# the code of an optimizer as in the cartpole example

import torch

# Define hyperparameters for the neural network
first_hidden_layer= 512;
second_hidden_layer= 128;

def create_network():

    # Define a deep neural network using Sequential:
    # Each layer feeds directly into the next one.
    dnn = torch.nn.Sequential(
        # First fully connected layer maps state inputs to 512 hidden units
        torch.nn.Linear(state_size[0], first_hidden_layer),

        # ReLU activation introduces nonlinearity
        torch.nn.ReLU(),
```

```
        # Second fully connected layer (hidden layer with 128 units)
        torch.nn.Linear(first_hidden_layer, second_hidden_layer),

        # Another ReLU activation
        torch.nn.ReLU(),

        # Output layer: one unit per possible action
        # Produces Q-values
        torch.nn.Linear(second_hidden_layer, action_size)
    )

    # Return the constructed model
    return dnn;
```

```
q = create_network();

learning_rate = 0.0001;
optimizer = torch.optim.RMSprop(q.parameters(), lr=learning_rate);
```

5 - Create a policy that is greedy with respect to the Q-function

```
# YOUR CODE HERE

# Write a function that, given a state, returns the action
# that maximizes the Q-function.

def nfq_pi(state):

    # convert state to a torch tensor
    state = torch.as_tensor(state, dtype=torch.float32)

    # compute Q-values from the network
    q_values = q(state).detach().numpy().squeeze()

    # select greedy action
    action = int(np.argmax(q_values))

    return action
```

6 - Add the code for the experience buffer

```
# YOUR CODE HERE

# Add the code for the experience buffer as in the cartpole example

# define the batch size
batch_size = 1024;

# define the structured dtype for an experience tuple
experience_type = np.dtype([
    ('state',      np.float32, state_size),    # current state
    ('action',     np.int8),                   # action taken
    ('reward',     np.float32),                # reward received
    ('next_state', np.float32, state_size),    # next state
    ('failure',    np.int8)                    # terminal flag (1 if done)
])
```

7 - Add the code of the exploration strategy based on epsilon-greedy method:

```
# YOUR CODE HERE

# Add the code for an exploration strategy based on the epsilon-greedy algorithm

import random;

# Exploration vs exploitation parameter
epsilon = 0.5

def epsilon_greedy(state):

    # Exploration
    if random.random() < epsilon:
        action = random_pi(state)

    # Exploitation
    else:
        action = nfq_pi(state)

    return action
```

8 - Add the code to train the neural network using the collected experiences:

```
# YOUR CODE HERE

# Re-use the optimize method as in the cartpole example

# define the discount factor
gamma = 0.99;

# define the number of optimization epochs
epochs = 40;

def optimize(batch):

    # convert experience fields to tensors
    states      = torch.from_numpy(batch['state'].copy()).float()
    actions     = torch.from_numpy(batch['action'].copy()).long()
    rewards     = torch.from_numpy(batch['reward'].copy()).float()
    next_states = torch.from_numpy(batch['next_state'].copy()).float()
    failures    = torch.from_numpy(batch['failure'].copy()).float()

    # perform multiple epochs of optimization
    for epoch in range(epochs):

        # compute target:  $r + \gamma \max_a Q(\text{next\_state}, a) * (1 - \text{done})$ 
        # remember to detach, we need to treat these values as constants!
        with torch.no_grad():
            q_next = q(next_states);
            max_q_next = q_next.max(dim=1).values;
            target = rewards + gamma * max_q_next * (1.0 - failures);

        # compute current Q(s, a)
        q_current_all = q(states);
        q_current = q_current_all.gather(1, actions.unsqueeze(1)).squeeze(1);

        # compute the Mean Squared TD Error loss
        td_error = target - q_current;

        # compute the loss
        loss = (td_error ** 2).mean();

        # optimize network parameters
        optimizer.zero_grad();
        loss.backward();
        optimizer.step();
```

```
return loss;
```

9 - Write the code of the NFQ algorithm

```
# YOUR CODE HERE

# Re-use the optimize method as in the cartpole example

def nfq(max_episodes):

    # create a batch index
    index = 0;

    # create a score tracker for statistic purposes
    scores = [];

    # create an empty experience batch
    experiences_batch = np.empty(batch_size, dtype=experience_type);

    # train until for the maximum number of episodes
    for episode in range(max_episodes):

        # reset the environment at the beginning of the episode
        state, _ = env.reset();
        done = False;

        # interact with the environment until the episode is done
        while not done:

            # select the action using the exploration policy
            action = epsilon_greedy(state);

            # perform the selected action
            next_state, reward, terminal, truncated, _ = env.step(action);
            done = terminal or truncated;
            failure = terminal and not truncated;

            # store the experience
            experiences_batch[index] = (state, action, reward, next_state, failure);
            index += 1;
```

```
    # if the experience collected so far is enough...
    if index == len(experiences_batch):
        # optimize the model
        optimize(experiences_batch);
        # and clear the buffer
        index = 0;
        experiences_batch = np.empty(shape=(len(experiences_batch),),
        ↪ dtype=experience_type);

        # update current state to next state
        state = next_state;

    # After each episode, evaluate the policy
    score = evaluate(nfq_pi, episodes=10);

    # store the score in the tracker
    scores.append(score);

    # print some informative logging
    message = 'Episode {:03}, score {:.05.1f}';
    message = message.format(episode+1, score);
    print(message, end='\r', flush=True);

return scores;
```

10 - Apply NFQ to the environment

```
# YOUR CODE HERE

# Apply NFQ to the acrobot environment

nfq(max_episodes = 3000);
```

Episode 3000, score -76.61

11 - Evaluate the learned policy

```
# YOUR CODE HERE

agent_score = evaluate(nfq_pi, episodes=100);
print("NFQ score", agent_score);
```


NFQ score -78.52

12 - Compare performance of random policy vs the one learned by the neural network

```
# YOUR CODE HERE

# Run the evaluation on the two policies and compare

agent_score = evaluate(nfq_pi, episodes=100);
random_score = evaluate(random_pi, episodes=100);

print("Random score", random_score);
print("Agent score", agent_score);
```

Random score -499.28

Agent score -76.65

13 - Show the policy behavior

```
# YOUR CODE HERE

# You can show the policy learned by the agent using the show_policy function

show_policy(nfq_pi, n_episodes=1)
```

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[27], line 5
      1 # YOUR CODE HERE
      2
      3 # You can show the policy learned by the agent using the show_policy function
----> 5 show_policy(nfq_pi, n_episodes=1)

Cell In[5], line 29, in show_policy(pi, n_episodes, max_steps)
     27 plt.axis("off")
     28 plt.title(f"Episode {episode+1}, Step {step}")
----> 29 plt.show()
     30 plt.close()
     32 # increment the step counter
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/pyplot.py:614, in show(*args, **kwargs)

```
570 """
571 Display all open figures.
572
( ... ) 611 explicitly there.
612 """
613 _warn_if_gui_out_of_main_thread()
--> 614 return _get_backend_mod().show(*args, **kwargs)
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib_inline/backend_inline.py:90, in show(close, block)

```
88 try:
89     for figure_manager in Gcf.get_all_fig_managers():
--> 90         display(
91             figure_manager.canvas.figure,
92             metadata=_fetch_figure_metadata(figure_manager.canvas.figure)
93         )
94 finally:
95     show._to_draw = []
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/IPython/core/display_functions.py:278, in display(include, exclude, metadata, trans

```
276     publish_display_data(data=obj, metadata=metadata, **kwargs)
277 else:
--> 278     format_dict, md_dict = format(obj, include=include, exclude=exclude)
279     if not format_dict:
280         # nothing to display (e.g. _ipython_display_ took over)
281         continue
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/IPython/core/formatters.py:238, in DisplayFormatter.format(self, obj, include, excl

```
236 md = None
237 try:
--> 238     data = formatter(obj)
239 except:
240     # FIXME: log the exception
```

```
241     raise
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/decorator.py:232, in decorate.<locals>.fun(*args, **kw)

```
230 if not kwsyntax:
231     args, kw = fix(args, kw, sig)
--> 232 return caller(func, *(extras + args), **kw)
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/IPython/core/formatters.py:282, in catch_format_error(method, self, *args, **kwargs)

```
280 """show traceback on failed format call"""
281 try:
--> 282     r = method(self, *args, **kwargs)
283 except NotImplementedError:
284     # don't warn on NotImplementedError
285     return self._check_return(None, args[0])
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/IPython/core/formatters.py:402, in BaseFormatter.__call__(self, obj)

```
400     pass
401 else:
--> 402     return printer(obj)
403 # Finally look for special method names
404 method = get_real_method(obj, self.print_method)
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/IPython/core/pylabtools.py:170, in print_figure(fig, fmt, bbox_inches, base64, **kwargs)

```
167     from matplotlib.backend_bases import FigureCanvasBase
168     FigureCanvasBase(fig)
--> 170 fig.canvas.print_figure(bytes_io, **kw)
171 data = bytes_io.getvalue()
172 if fmt == 'svg':
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/backend_bases.py:2155, in FigureCanvasBase.print_figure(self, filename,

```
2152     # we do this instead of `self.figure.draw_without_rendering`
2153     # so that we can inject the orientation
2154     with getattr(renderer, "_draw_disabled", nullcontext)():
```

```
-> 2155         self.figure.draw(renderer)
    2156 if bbox_inches:
    2157     if bbox_inches == "tight":
```

```
File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-
packages/matplotlib/artist.py:94, in _finalize_rasterization.<locals>.draw_wrapper(artist,
    92 @wraps(draw)
    93 def draw_wrapper(artist, renderer, *args, **kwargs):
--> 94     result = draw(artist, renderer, *args, **kwargs)
    95     if renderer._rasterizing:
    96         renderer.stop_rasterizing()
```

```
File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-
packages/matplotlib/artist.py:71, in allow_rasterization.<locals>.draw_wrapper(artist, rend
    68     if artist.get_agg_filter() is not None:
    69         renderer.start_filter()
--> 71     return draw(artist, renderer)
    72 finally:
    73     if artist.get_agg_filter() is not None:
```

```
File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-
packages/matplotlib/figure.py:3257, in Figure.draw(self, renderer)
    3254         # ValueError can occur when resizing a window.
    3256     self.patch.draw(renderer)
-> 3257     mimage._draw_list_compositing_images(
    3258         renderer, self, artists, self.suppressComposite)
    3260     renderer.close_group('figure')
    3261 finally:
```

```
File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-
packages/matplotlib/image.py:134, in _draw_list_compositing_images(renderer, parent, artist
    132 if not_composite or not has_images:
    133     for a in artists:
--> 134         a.draw(renderer)
    135 else:
    136     # Composite any adjacent images together
    137     image_group = []
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/artist.py:71, in allow_rasterization.<locals>.draw_wrapper(artist, render

```
    68     if artist.get_agg_filter() is not None:
    69         renderer.start_filter()
--> 71     return draw(artist, renderer)
    72 finally:
    73     if artist.get_agg_filter() is not None:
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/axes/_base.py:3180, in _AxesBase.draw(self, renderer)

```
    3177     for spine in self.spines.values():
    3178         artists.remove(spine)
-> 3180 self._update_title_position(renderer)
    3182 if not self.axison:
    3183     for _axis in self._axis_map.values():
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/axes/_base.py:3124, in _AxesBase._update_title_position(self, renderer)

```
    3122 if title.get_text():
    3123     for ax in axs:
-> 3124         ax.yaxis.get_tightbbox(renderer) # update offsetText
    3125         if ax.yaxis.offsetText.get_text():
    3126             bb = ax.yaxis.offsetText.get_tightbbox(renderer)
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/axis.py:1351, in Axis.get_tightbbox(self, renderer, for_layout_only)

```
    1349 if renderer is None:
    1350     renderer = self.get_figure(root=True)._get_renderer()
-> 1351 ticks_to_draw = self._update_ticks()
    1353 self._update_label_position(renderer)
    1355 # go back to just this axis's tick labels
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/axis.py:1283, in Axis._update_ticks(self)

```
    1281 major_locs = self.get_majorticklocs()
    1282 major_labels = self.major.formatter.format_ticks(major_locs)
-> 1283 major_ticks = self.get_major_ticks(len(major_locs))
    1284 for tick, loc, label in zip(major_ticks, major_locs, major_labels):
```

```
1285     tick.update_position(loc)
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/axis.py:1664, in Axis.get_major_ticks(self, numticks)

```
1660     numticks = len(self.get_majorticklocs())
1662     while len(self.majorTicks) < numticks:
1663         # Update the new tick label properties from the old.
-> 1664         tick = self._get_tick(major=True)
1665         self.majorTicks.append(tick)
1666         self._copy_tick_props(self.majorTicks[0], tick)
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/axis.py:1592, in Axis._get_tick(self, major)

```
1588     raise NotImplementedError(
1589         f"The Axis subclass {self.__class__.__name__} must define "
1590         "_tick_class or reimplement _get_tick()")
1591     tick_kw = self._major_tick_kw if major else self._minor_tick_kw
-> 1592     return self._tick_class(self.axes, 0, major=major, **tick_kw)
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/axis.py:429, in YTick.__init__(self, *args, **kwargs)

```
428     def __init__(self, *args, **kwargs):
-> 429         super().__init__(*args, **kwargs)
430         # x in axes coords, y in data coords
431         ax = self.axes
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/axis.py:117, in Tick.__init__(self, axes, loc, size, width, color, tickdir)

```
114     self._width = width
116     if color is None:
-> 117         color = mpl.rcParams[f"{name}.color"]
119     if pad is None:
120         pad = mpl.rcParams[f"{name}.{major_minor}.pad"]
```

File /opt/homebrew/Caskroom/miniconda/base/envs/machine-learning/lib/python3.13/site-packages/matplotlib/__init__.py:778, in RcParams.__getitem__(self, key)

```
773     except KeyError as err:
774         raise KeyError(
```

```

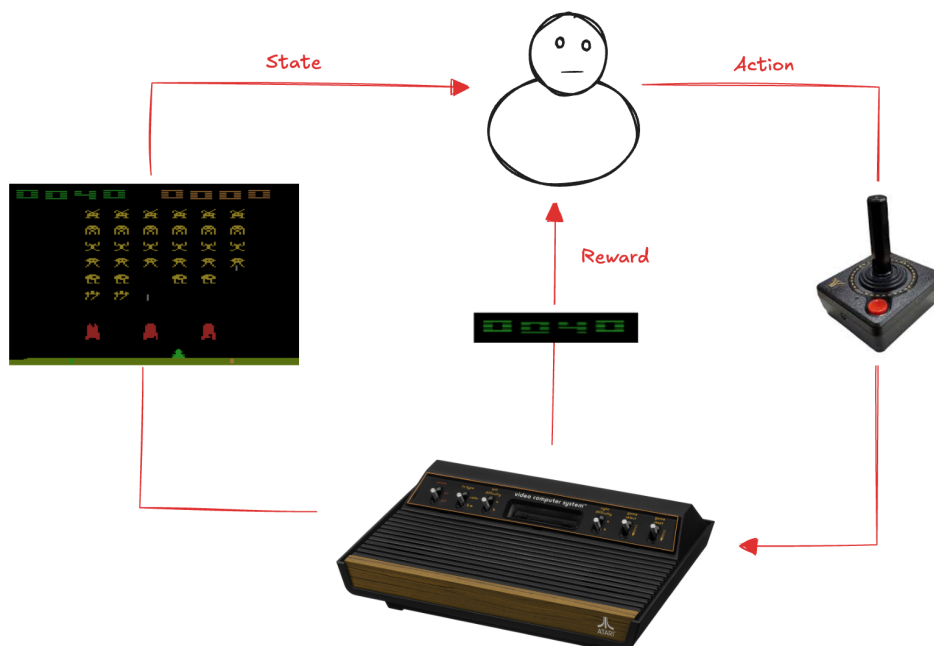
775         f"{key} is not a valid rc parameter (see rcParams.keys() for "
776         f"a list of valid parameters)") from err
--> 778 def __getitem__(self, key):
779     if key in _deprecated_map:
780         version, alt_key, alt_val, inverse_alt = _deprecated_map[key]

```

KeyboardInterrupt:

4.2 Atari Games

Atari 2600 is a video game console which provides several popular games such as pong, space invaders, pacman, break out, centipede and many more. Gymnasium includes ALE-py, the **Arcade Learning Environment (ALE)** which is a simple framework that allows researchers to develop AI agents for Atari 2600 games. It is built on top of the **Stella emulator** and separates the details of emulation from agent design. The main Gymnasium page provides a list of all the Atari environments. The objective of this notebook is to **train an agent to play a game of the Atari console** using the DQN algorithm **learning from raw pixel data**, like in the famous paper V. Mnih, et al. **"Playing Atari with Deep Reinforcement Learning"**, NIPS 2013:



In particular, we can consider **the classic Pong game**. The game is played like tennis. Using a controller, each player hits the ball by moving vertically the paddles on the game field. A player scores one point when the opponent misses a hit. The first player who scores 21 points wins

the game. The action space is discrete and deterministic with 6 actions (not all meaningful for Pong): 0 (noop), 1 (fire), 2 (right) and 3 (left), 4 (right fire), 5 (left fire) The observation space is the screen image (210x160x3). The state space is really high-dimensional and to use it you need to modify the neural network architecture adding some convolutional layer in order to extract features from the image.

1 - Define a variable with the name of the Atari game you want to solve (e.g. Pong) and create an instance of the environment.

```
# YOUR CODE HERE

# Import Gymnasium and create an instance with "render_mode" set to "rgb_array". You
# need to import also "ale_py" plugin and to register it with Gymnasium
↳ (gym.register_envs(ale_py)).
# Set also the parameter "frameskip" to 1. Later, we will see that we want to control
# the number of frames to skip as a parameter of our code.

import gymnasium as gym
import ale_py
gym.register_envs(ale_py)

env = gym.make('ALE/Pong-v5', render_mode="rgb_array", frameskip=1)

state_size = env.observation_space.shape;
action_size = env.action_space.n;

print('State Space: ', state_size);
print('Action Space: ', action_size);
```

State Space: (210, 160, 3)

Action Space: 6

2 - Create a function to show the game in action by rendering the environment and selecting actions at random or using a policy.

```
# YOUR CODE HERE

# Define a function "random_pi" that takes as input a state and a Q-function and
# returns a random action.

import numpy as np
```



```
def random_pi(state):
    # selects an action uniformly at random
    # from the environment's action space.
    return env.action_space.sample()

# YOUR CODE HERE

# Define a function "show_policy" that takes as input an environment,
# a policy and a Q-function and shows the policy in action for a number of episodes

import matplotlib.pyplot as plt
from IPython.display import clear_output

def show_policy(pi, n_episodes=1, max_steps=500):

    # start the episodes
    for episode in range(n_episodes):

        # reset the environment
        state, _ = env.reset();
        done = False;

        # initialize the step counter
        step = 0;

        # run the episode until done or max_steps
        while not done and step < max_steps:

            # make a step
            action = pi(state);
            state, _, terminated, truncated, _ = env.step(action);
            done = terminated or truncated;

            # render the environment
            clear_output(wait=True)
            plt.imshow(env.render())
            plt.axis("off")
            plt.title(f"Episode {episode+1}, Step {step}")
            plt.show()
            plt.close()
```

```
# increment the step counter
step += 1;
```

3 - Use the previous function in order to see one episode of the game with a random policy.

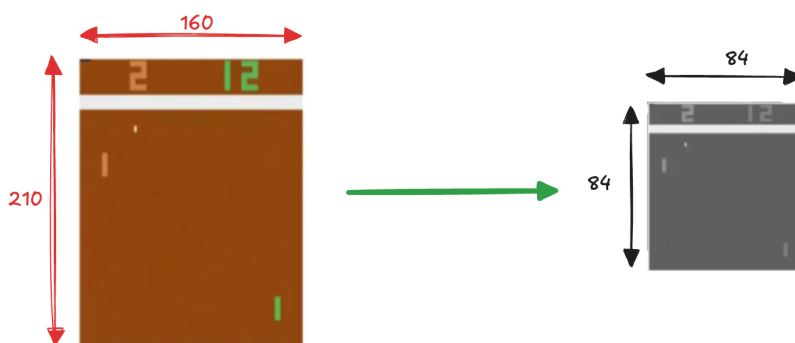
```
# YOUR CODE HERE

# Just call the "show_policy" function defined above passing the environment, None as
↳ agent and
# the number of episodes you want to visualize

show_policy(random_pi, n_episodes=1, max_steps=500);
```

4 - We would like to use as state the raw pixels data. However, **directly feeding the raw game screen image is not efficient**, since it will be **computationally expensive**. So we need to pre-process the environment output to reduce the dimensionality of the problem. Moreover the Atari environments have some game specific issues, which are not relevant for the learning process, that we need to manage. In order to do that, you can use the **wrapper function**, which is a convenient way to modify an existing environment without having to alter the underlying code directly. Using wrappers will allow you to avoid a lot of boilerplate code and make our environment more modular. Here a list of the useful modifications that we can apply to the Atari environment:

1) The raw game screen size is 210x160 pixels with three color channels (RGB), however we don't need the color information and we can resize the image in order to reduce the dimensionality of the problem.



2) Atari games typically run at 60 frames per second, however not every frame contains crucial information for decision-making. To reduce computation and memory requirements, and to speed up training, we can skip a certain number of frames. Moreover, some games have a flickering effect (when the game draws different portions of the screen on even and odd frames) due

to the old Atari platform's limitation. For the human eye, such quick changes are not visible, but they can confuse a neural network. So, we can take the maximum of every pixel in the last two frames and use it as observation.

3) In order to inject variability into the initial state of each episode, promoting exploration and helping the agent discover a more robust policy across different starting conditions, we can introduce a certain number of "no-operation" actions at the beginning of each episode in order to add stochasticity to the initial conditions of the environment.

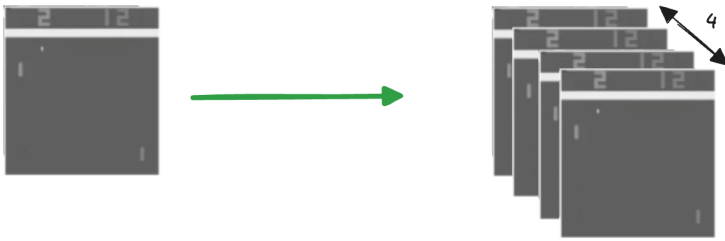
Add a wrapper in order to apply all those modifications to the environment.

```
# YOUR CODE HERE

# Gymnasium provides a wrapper to apply those pre-processing to the
# observations for the Atari games, you can use the "AtariPreprocessing" wrapper
# from "gymnasium.wrappers" module.

env = gym.wrappers.AtariPreprocessing(env,
                                     noop_max = 30,           # the max number no-ops
                                     ↪ actions are taken at reset
                                     frame_skip = 4,           # the number of frames
                                     ↪ between new observation
                                     screen_size = 84,          # resize the image frame
                                     grayscale_obs = True,       # convert the image frame
                                     ↪ to grayscale
                                     scale_obs = True)          # scale the image frame
```

5 - A single raw screenshot of the game used as state is not enough to extract the information about the game state. How can the agent learn to play just seeing a single image at a time? It is **not possible to build an idea about how the game objects are moving**. In Pong, just looking at a single frame don't provide enough information to understand if the ball is moving left to right or right to left. For example, in the Cart-pole example, the state is composed by the position of the cart, the angle of the pole, but it includes also the velocity of both. To deal with this, we need to consider as state a series of frame and not just one. In this way, the agent can learn how the objects are moving. So, we need to apply a wrapper that will stack the last N (e.g. 4) frames together and use them as state.



```
# YOUR CODE HERE

# Gymnasium provides the "FrameStackObservation" wrapper to stack the last n frames
# of the game. Apply it to the
# environment.

env = gym.wrappers.FrameStackObservation(env, 4);
```

6 - Some games, as Pong, require a user to press the FIRE button to start the game, we need to take this action on reset. Add another wrapper to take this action on reset.

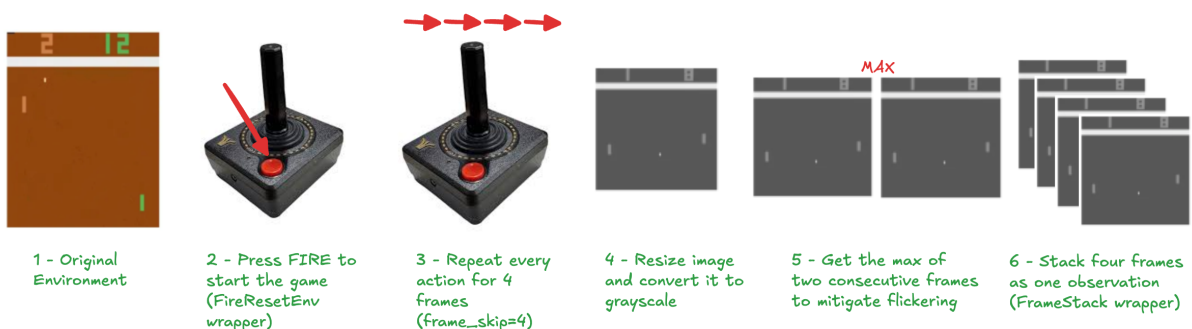
```
# YOUR CODE HERE

# This wrapper can be found in the stable_baselines3 library. It is called
# "FireResetEnv"
# Import it and apply it to the environment.

import stable_baselines3

env = stable_baselines3.common.atari_wrappers.FireResetEnv(env);
```

7 - Finally, we write a "make_atari_env()" function that takes as input the name of the environment and returns the wrapped environment



```
# YOUR CODE HERE

# You create an environment and then apply in order all the wrappers needed

def make_atari_env(env_name, mode="rgb_array", seed=None):

    # create the environment
    env = gym.make(env_name, render_mode=mode, frameskip=1);

    if seed is not None:
        env.np_random = np.random.Generator(np.random.PCG64(seed));

    # apply the fire reset wrapper
    env = stable_baselines3.common.atari_wrappers.FireResetEnv(env);

    # apply a wrapper to skip frames, scale the observation and convert it to grayscale
    env = gym.wrappers.AtariPreprocessing(env, noop_max=30, frame_skip=4,
    ↪ screen_size=84, grayscale_obs=True, scale_obs=True);

    # apply a wrapper to stack the frames
    env = gym.wrappers.FrameStackObservation(env, 4);

    # return the environment
    return env;

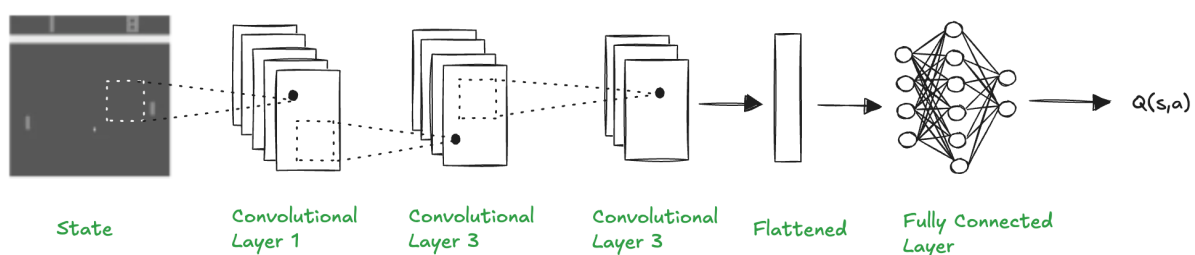
# set the seed for reproducibility
seed = 42;

# set the name of the Atari environment
env_name = 'ALE/Pong-v5';

# create the environment
env = make_atari_env(env_name, seed=seed);
state_size = env.observation_space.shape;
action_size = env.action_space.n;
```

8 - Since we are dealing with images, **instead of using a fully connected neural network, this time we need to use a convolutional neural network**. The convolutional layers extract the features from the image and output the feature maps, then the flattened feature map can be fed to the fully connected layer which returns the Q value. You can implement the following architecture:

- Convolutional layer with 32 filters, kernel size 8, stride 4, padding 0, followed by a ReLU activation function
- Convolutional layer with 64 filters, kernel size 4, stride 2, padding 0, followed by a ReLU activation function
- Convolutional layer with 64 filters, kernel size 3, stride 1, padding 0, followed by a ReLU activation function
- Flatten layer
- Fully connected layer with 512 units, followed by a ReLU activation function
- Fully connected layer with a number of units equal to the number of actions



YOUR CODE HERE

Add the code for the neural network using the PyTorch library. And
create two networks, one for the online network and one for the target network.

```
import torch
```

```
def create_network():
    dnn = torch.nn.Sequential(
        torch.nn.Conv2d(state_size[0], 32, kernel_size=8, stride=4),
        torch.nn.ReLU(),
        torch.nn.Conv2d(32, 64, kernel_size=4, stride=2),
        torch.nn.ReLU(),
        torch.nn.Conv2d(64, 64, kernel_size=3, stride=1),
        torch.nn.ReLU(),

        # flatten the output
        torch.nn.Flatten(),

        # fully connected layers
        torch.nn.Linear(7 * 7 * 64, 512),
        torch.nn.ReLU(),
        torch.nn.Linear(512, action_size)
```

```
);

return dnn;
```

```
online_q = create_network();
target_q = create_network();
```

```
learning_rate = 0.007;
optimizer = torch.optim.RMSprop(online_q.parameters(), lr=learning_rate);
```

```
def update_target():
    # copy the parameters from the online model to the target model
    for target, online in zip(target_q.parameters(), online_q.parameters()):
        target.data.copy_(online.data)
```

```
def dqn_pi(state):
    # convert the state into a tensor
    state = np.array(state);
    state = torch.from_numpy(state).unsqueeze(0);

    # calculate q_values from the network
    q_values = online_q(state).detach();

    # act greedy
    action = np.argmax(q_values).data.numpy();

    return action;
```

9 - Add the Replay Memory to store the transitions (state, action, reward, next_state, done).

```
# YOUR CODE HERE

# Add the code for the replay buffer already used presented in the DQN implementation.
# We need a larger replay buffer because the problem is more complex.

replay_memory_size = 100000;

# define the structured dtype for an experience tuple
experience_type = np.dtype([
```

```
    ('state',      np.float32, state_size),    # current state
    ('action',     np.int8),                  # action taken
    ('reward',     np.float32),              # reward received
    ('next_state', np.float32, state_size),    # next state
    ('failure',    np.int8)                  # terminal flag (1 if done)
])
```

```
# Set the replay memory size
```

```
memory_size = 100000;
```

```
# Create the replay memory
```

```
replay_memory = {
    'size': memory_size,
    'buffer': np.empty(shape=(memory_size,), dtype=experience_type),
    'index': 0,
    'entries': 0
}
```

```
def store_experience(experience):
```

```
    # store the experience in the buffer
```

```
    replay_memory['buffer'][replay_memory['index']] = experience;
```

```
    # update the number of experiences in the buffer
```

```
    replay_memory['entries'] = min(replay_memory['entries'] + 1,
```

```
    ↪ replay_memory['size']);
```

```
    # update index, if the memory is full, start from the begging
```

```
    replay_memory['index'] += 1;
```

```
    replay_memory['index'] = replay_memory['index'] % replay_memory['size'];
```

```
# Set the batch size for sampling experiences
```

```
batch_size = 32;
```

```
# function to sample a batch of experiences from the replay memory
```

```
def sample_experiences():
```

```
    # select uniformly at random a batch of experiences from the memory
```

```
    idxs = np.random.choice(range(replay_memory['entries']), batch_size,
```

```
    ↪ replace=False);
```

```
    # return the batch of experiences
```



```
experiences = replay_memory['buffer'][idxs];

return experiences;
```

10- Create a the code for exploration (decaying epsilon-greedy policy) and exploitation (greedy policy) and the code for doing the evaluation of the agent.

```
# YOUR CODE HERE

# Add the code for decaying epsilon and the epsilon-greedy policy as already
# done for the DQN implementation

# define decay parameters (max, min, steps)
epsilon_max = 1.0;
epsilon_min = 0.01;
epsilon_decay_steps = 10000;

# generate epsilons
epsilons = np.logspace(start=0, stop=-2, num=epsilon_decay_steps, base=10)

# normalize epsilons
epsilons = (epsilons - epsilon_min) / (epsilon_max - epsilon_min);

# scale epsilons to the desired range
epsilons = (epsilon_max - epsilon_min) * epsilons + epsilon_min;
```

```
import random

def epsilon_greedy(state, step):
    # get the epsilon value
    epsilon = epsilons[step] if step < epsilon_decay_steps else epsilon_min;

    # Exploration
    if random.random() < epsilon:
        action = random_pi(state)

    # Exploitation
    else:
        action = dqn_pi(state)

    return action;
```

```
# YOUR CODE HERE

# Add the code for an evaluation or the agent as already
# done for the DQN implementation.

def evaluate(pi, episodes=1):

    # collect total rewards per episode
    rewards = [];

    # loop over episodes
    for episode in range(episodes):

        # reset the environment
        state, _ = env.reset();
        done = False;
        total_reward = 0.0;

        # run an episode
        while not done:
            action = pi(state);
            state, reward, terminal, truncated, _ = env.step(action)
            total_reward += reward;
            done = terminal or truncated;

        # store the total reward
        rewards.append(total_reward);

    # return the average reward over the episodes
    return np.mean(rewards);
```

11 - Add the code for the optimization of the Q-function based on the experience sampled from the replay buffer. Remember to use the target network to compute the target Q-values.

```
# YOUR CODE HERE

# Add the code of the optimization algorithm as already
# done for the DQN implementation. This time you need to pay attention
# to states which are a stack of four images

# define the discount factor
gamma = 0.99;
```

```
def optimize():

    # sample a batch of experiences
    batch = sample_experiences();

    # prepare the experience as tensors
    states      = torch.from_numpy(batch['state'].copy()).float();
    actions     = torch.from_numpy(batch['action'].copy()).long();
    rewards     = torch.from_numpy(batch['reward'].copy()).float();
    next_states = torch.from_numpy(batch['next_state'].copy()).float();
    failures    = torch.from_numpy(batch['failure'].copy()).float();

    # get the values of the Q-function at next state from the "target" network
    # remember to detach, we need to treat these values as constants
    q_target_next = target_q(next_states).detach();

    # get the max value
    max_q_target_next = q_target_next.max(1)[0];

    # one important step, often overlooked, is to ensure
    # that failure states are grounded to zero
    max_q_target_next *= (1 - failures.float())

    # calculate the target
    target = rewards + gamma * max_q_target_next;

    # finally, we get the current estimate of Q(s,a)
    # here we query the current "online" network
    q_online_current = torch.gather(online_q(states), 1,
    ↪ actions.unsqueeze(1)).squeeze(1);

    # create the errors
    td_error = target - q_online_current;

    # calculate the loss
    loss = td_error.pow(2).mean();

    # backward pass: compute the gradients
    optimizer.zero_grad();
    loss.backward();

    # update model parameters
```

```
optimizer.step();
```

12 - Write the code of the DQN algorithm (exploiting the new convolutional network architecture). You should modify the code in order to save the network weights after some episodes.

```
# YOUR CODE HERE

# Add the code for the DQN algorithm, exploiting the convolutional neural architecture
# already defined, the replay buffer as already implemented in DQN algorithm.
# At the beginning, if a model file is already present on the disk, you should load the
# weights and
# sometimes, you should save the model on the disk.

# define the size of the replay memory before starting the training
memory_start_size = 1000;

# define the number of steps between target network updates
target_update_steps = 10;

def dqn(max_episodes):

    # create a score tracker for statistic purposes
    scores = [];

    # counter for the number of steps
    step = 0;

    # update the target model with the online one
    update_target();

    # train until the maximum number of episodes
    for episode in range(max_episodes):

        # reset the environment before starting the episode
        state, _ = env.reset();
        done = False;

        # interact with the environment until the episode is done
        while not done:

            # select the action using the exploration policy
            action = epsilon_greedy(state, step);
```

```
# perform the selected action
next_state, reward, terminal, truncated, _ = env.step(action);
done = terminal or truncated;
failure = terminal and not truncated;

# store the experience into the replay buffer
experience = (state, action, reward, next_state, failure);
store_experience(experience);

# optimize the online model after the replay buffer is large enough
if replay_memory['entries'] > memory_start_size:
    optimize();

    # sometimes, synchronize the target model with the online model
    if step % target_update_steps == 0:
        update_target();

# update current state to next state
state = next_state;

# update the step counter
step += 1;

# After each episode, evaluate the policy
score = evaluate(dqn_pi, episodes=10);

# store the score in the tracker
scores.append(score);

# print some informative logging
message = 'Episode {:03}, score {:.05.1f}';
message = message.format(episode+1, score);
print(message, end='\n', flush=True);

return scores;
```

14 - Apply the algorithm to the ATARI environment. Pay attention, the following code takes a lot of time to run (days on a powerful machine).

```
# YOUR CODE HERE

# Apply the previous written algorithm to the ATARI environment, please pay attention
# to the parameters.

# run the DQN algorithm
scores = dqn(max_episodes=100);
```

Episode 009, score -21.0

15 - Smooth the result using a sliding window

```
# YOUR CODE HERE

# smooth the result using a sliding window
sliding_windows = 20
scores = np.convolve(scores, np.ones(sliding_windows)/sliding_windows, mode='valid');
```

16 - Show the score curve as a function of the number of episodes

```
# YOUR CODE HERE

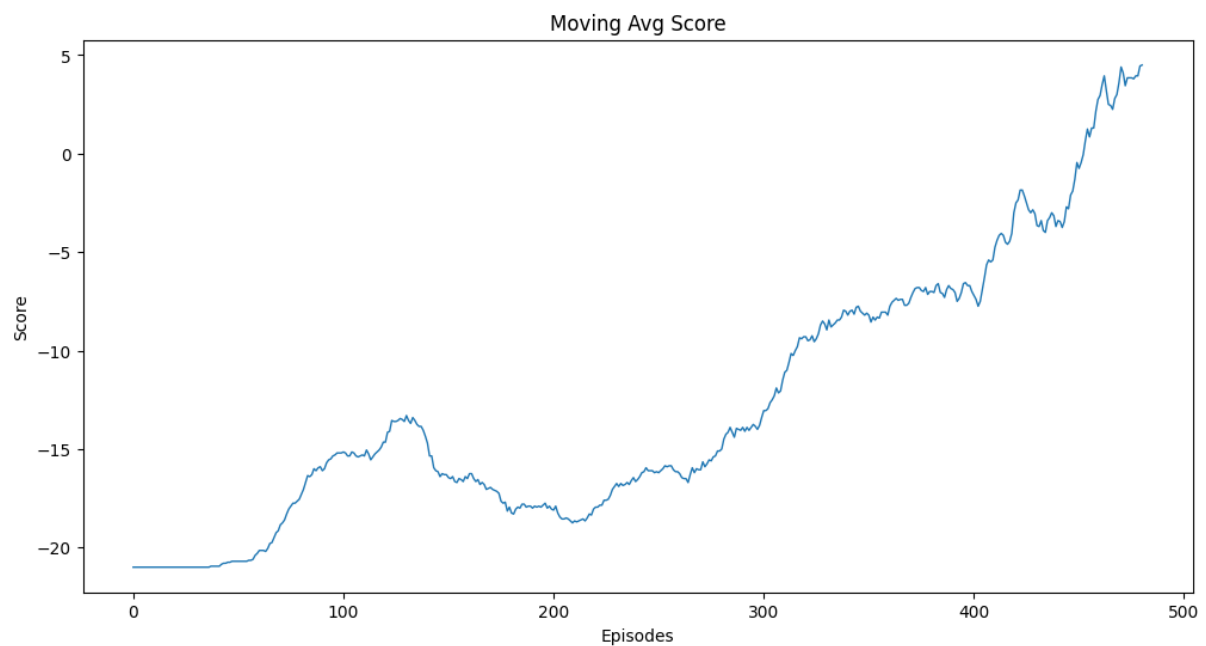
# Show the moving average reward during the training process

import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))

plt.plot(scores, linewidth=1)
plt.title('Moving Avg Score')
plt.ylabel('Score')
plt.xlabel('Episodes')

plt.show()
```



18 - Show the agent playing the game using the greedy policy

```
# YOUR CODE HERE

# You can show the policy learned by the agent using the "show_policy" function

show_policy(dqn_pi, n_episodes=1, max_steps=2000);
```

